
Особенности использования автоэнкодеров и вариационных автоэнкодеров для рекомендаций. Имплементации

Лашинин Олег

18.03.25

План занятия

- Автоэнкодеры в рекомендациях (идея + результаты)
- Обзорю про особенности использования
- Конкретные имплементации

Почему сегодня эта тема?



Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches

Maurizio Ferrari Dacrema
Politecnico di Milano, Italy
maurizio.ferrari@polimi.it

Paolo Cremonesi
Politecnico di Milano, Italy
paolo.cremonesi@polimi.it

Dietmar Jannach
University of Klagenfurt, Austria
dietmar.jannach@aau.at

ABSTRACT

Deep learning techniques have become the method of choice for researchers working on algorithmic aspects of recommender systems. With the strongly increased interest in machine learning in general, it has, as a result, become difficult to keep track of what represents the state-of-the-art at the moment, e.g., for top-n recommendation tasks. At the same time, several recent publications point out problems in today’s research practice in applied machine learning, e.g., in terms of the reproducibility of the results or the choice of the baselines when proposing new models.

In this work, we report the results of a systematic analysis of algorithmic proposals for top-n recommendation tasks. Specifically, we considered 18 algorithms that were presented at top-level research conferences in the last years. Only 7 of them could be reproduced with reasonable effort. For these methods, it however turned out that 6 of them can often be outperformed with comparably simple heuristic methods, e.g., based on nearest-neighbor or graph-based techniques. The remaining one clearly outperformed the baselines but did not consistently outperform a well-tuned non-neural linear ranking method. Overall, our work sheds light on a number of potential problems in today’s machine learning scholarship and calls for improved scientific practices in this area.

CCS CONCEPTS

• Information systems → Collaborative filtering; Recommender systems; • General and reference → Evaluation.

KEYWORDS

Recommender Systems; Deep Learning; Evaluation; Reproducibility

ACM Reference Format:

Maurizio Ferrari Dacrema, Paolo Cremonesi, and Dietmar Jannach. 2019. Are We Really Making Much Progress? A Worrying Analysis of Recent Neural Recommendation Approaches. In *Thirteenth ACM Conference on Recommender Systems (RecSys ’19)*, September 16–20, 2019, Copenhagen, Denmark. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3298689.3347058>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

RecSys ’19, September 16–20, 2019, Copenhagen, Denmark

© 2019 Association for Computing Machinery.

ACM ISBN 978-1-4503-6243-6/19/09...\$15.00

<https://doi.org/10.1145/3298689.3347058>

1 INTRODUCTION

Within only a few years, deep learning techniques have started to dominate the landscape of algorithmic research in recommender systems. Novel methods were proposed for a variety of settings and algorithmic tasks, including top-n recommendation based on long-term preference profiles or for session-based recommendation scenarios [36]. Given the increased interest in machine learning in general, the corresponding number of recent research publications, and the success of deep learning techniques in other fields like vision or language processing, one could expect that substantial progress resulted from these works also in the field of recommender systems. However, indications exist in other application areas of machine learning that the achieved progress—measured in terms of accuracy improvements over existing models—is not always as strong as expected.

Lin [25], for example, discusses two recent neural approaches in the field of information retrieval that were published at top-level conferences. His analysis reveals that the new methods do *not* significantly outperform existing baseline methods when these are carefully tuned. In the context of recommender systems, an in-depth analysis presented in [29] shows that even a very recent neural method for session-based recommendation can, in most cases, be outperformed by very simple methods based, e.g., on nearest-neighbor techniques. Generally, questions regarding the true progress that is achieved in such applied machine learning settings are not new, nor tied to research based on deep learning. Already in 2009, Armstrong et al. [2] concluded from an analysis in the context of ad-hoc retrieval tasks that, despite many papers being published, the reported improvements “don’t add up”.

Different factors contribute to such phenomena, including (i) weak baselines; (ii) establishment of weak methods as new baselines; and (iii) difficulties in comparing or reproducing results across papers. One first problem lies in the choice of the baselines that are used in the comparisons. Sometimes, baselines are chosen that are too weak in general for the given task and dataset, and sometimes the baselines are not properly fine-tuned. Other times, baselines are chosen from the same family as the newly proposed algorithm, e.g., when a new deep learning algorithm is compared only against other deep learning baselines. This behaviour enforces the propagation of weak baselines. When previous deep learning algorithms were evaluated against too weak baselines, the new deep learning algorithm will not necessarily improve over strong non-neural baselines. Furthermore, with the constant flow of papers being published in recent years, keeping track of what represents a state-of-the-art baseline becomes increasingly challenging.

Besides issues related to the baselines, an additional challenge is that researchers use various types of datasets, evaluation protocols, performance measures, and data preprocessing steps, which makes

provided test split is much higher (0.92), which means that the distribution has a much higher popularity bias than a random split.

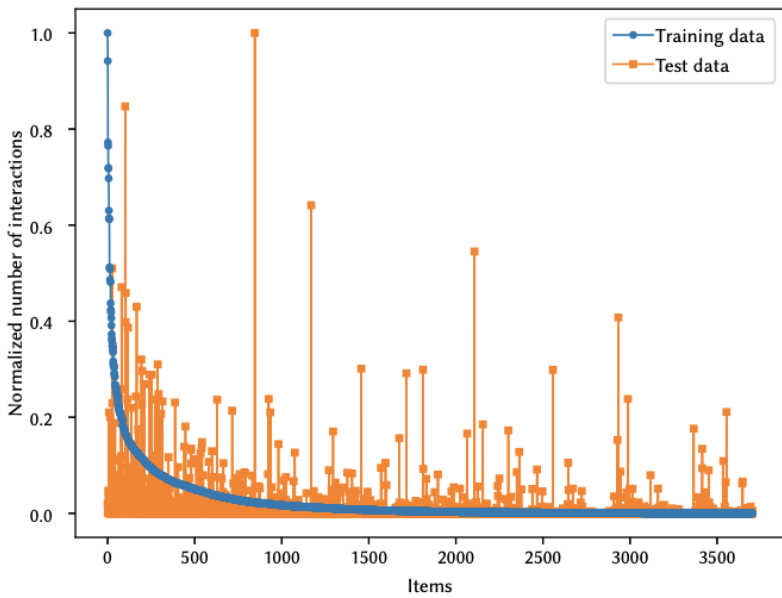


Figure 1: Popularity distributions of the provided training and test splits. In case of a random split, the normalized values should, on average, be close for both splits.

3.7 Variational Autoencoders for Collaborative Filtering (Mult-VAE)

Mult-VAE [24] is a collaborative filtering method for implicit feedback based on variational autoencoders. The work was presented at WWW ’18. With Mult-VAE, the authors introduce a generative model with multinomial likelihood, propose a different regularization parameter for the learning objective, and use Bayesian inference for parameter estimation. They evaluate their method on three binarized datasets that originally contain movie ratings or song play counts. The baselines in the experiments include both a matrix factorization method from 2008 [18], a linear model from 2011 [33], and a more recent neural method [51]. According to the

Table 8: Experimental results for Mult-VAE (Netflix data), using metrics and cutoffs reported in the original paper.

	REC@20	REC@50	NDCG@100
TopPop	0.0782	0.1643	0.1570
ItemKNN CF	0.2088	0.3386	0.3086
$P^3\alpha$	0.1977	0.3346	0.2967
$RP^3\beta$	0.2196	0.3560	0.3246
SLIM	0.2551	0.3995	0.3745
Mult-VAE	0.2626	0.4138	0.3756

The differences between Mult-VAE and SLIM in terms of the NDCG, the optimization goal, are quite small. In terms of the Recall, however, Mult-VAE improvements over SLIM seem solid. Since the choice of the used cutoffs (20 and 50 for Recall, and 100 for NDCG) is not very consistent in [24], we made additional measurements at different cutoff lengths. The results are provided in Table 9. They show that when using the NDCG as an optimization goal *and* as a performance measure, the differences between SLIM and Mult-VAE disappear on this dataset, and SLIM is actually sometimes slightly better. A similar phenomenon can be observed for the MovieLens dataset. In this particular case, therefore, the progress that is achieved through the neural approach is only partial and depends on the chosen evaluation measure.

Table 9: Experimental results for Mult-VAE using additional cutoff lengths for the Netflix dataset.

	NDCG@20	NDCG@50	REC@100	NDCG@100
SLIM	0.2473	0.3196	0.5289	0.3745
Mult-VAE	0.2448	0.3192	0.5476	0.3756

Почему сегодня эта тема?

Revisiting the Performance of iALS on Item Recommendation Benchmarks

Steffen Rendle*
srendle@google.com

Walid Krichene*
walidk@google.com

Li Zhang*
liqzhang@google.com

Yehuda Koren†
yehuda@google.com

Table 3: Quality results on the ML20M benchmark sorted by Recall@20 scores.

Method	Recall@20	Recall@50	NDCG@100	Result from
RecVAE [26]	0.414	0.553	0.442	[26]
H+Vamp (Gated) [14]	0.413	0.551	0.445	[14]
RaCT [19]	0.403	0.543	0.434	[19]
Mult-VAE [18]	0.395	0.537	0.426	[18]
LambdaNet [4]	0.395	0.534	0.427	[26]
iALS	0.395	0.532	0.425	our result
EASE [27]	0.391	0.521	0.420	[27]
CDAE [29]	0.391	0.523	0.418	[18]
Mult-DAE [18]	0.387	0.524	0.419	[18]
SLIM [20]	0.370	0.495	0.401	[18]
iALS	0.363	0.502	0.393	prev. result [13]
iALS	0.360	0.498	0.386	prev. result [18]
WARP [28]	0.314	0.466	0.341	[26]
Popularity	0.162	0.235	0.191	[27]

Почему сегодня эта тема?

Variational Autoencoders for Collaborative Filtering

Dawen Liang
Netflix
Los Gatos, CA
dliang@netflix.com

Rahul G. Krishnan
MIT
Cambridge, MA
rahulgk@mit.edu

Matthew D. Hoffman
Google AI
San Francisco, CA
mhoffman@google.com

Tony Jebara
Netflix
Los Gatos, CA
tjebara@netflix.com



arXiv

<https://arxiv.org> › stat

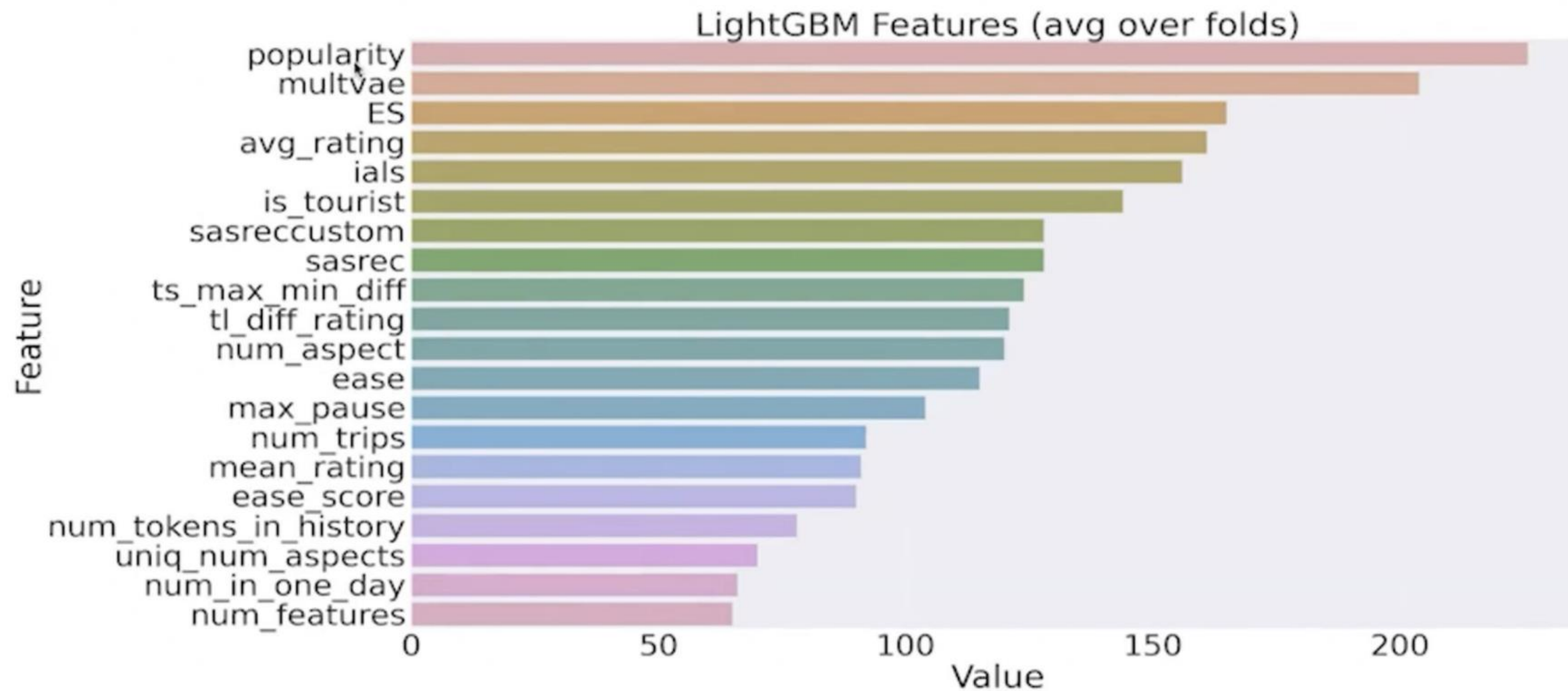


Variational Autoencoders for Collaborative Filtering

by D Liang · 2018 · Cited by 1517 — We extend **variational autoencoders** (VAEs) to **collaborative filtering** for implicit feedback. This non-linear probabilistic model enables us to go beyond the ...

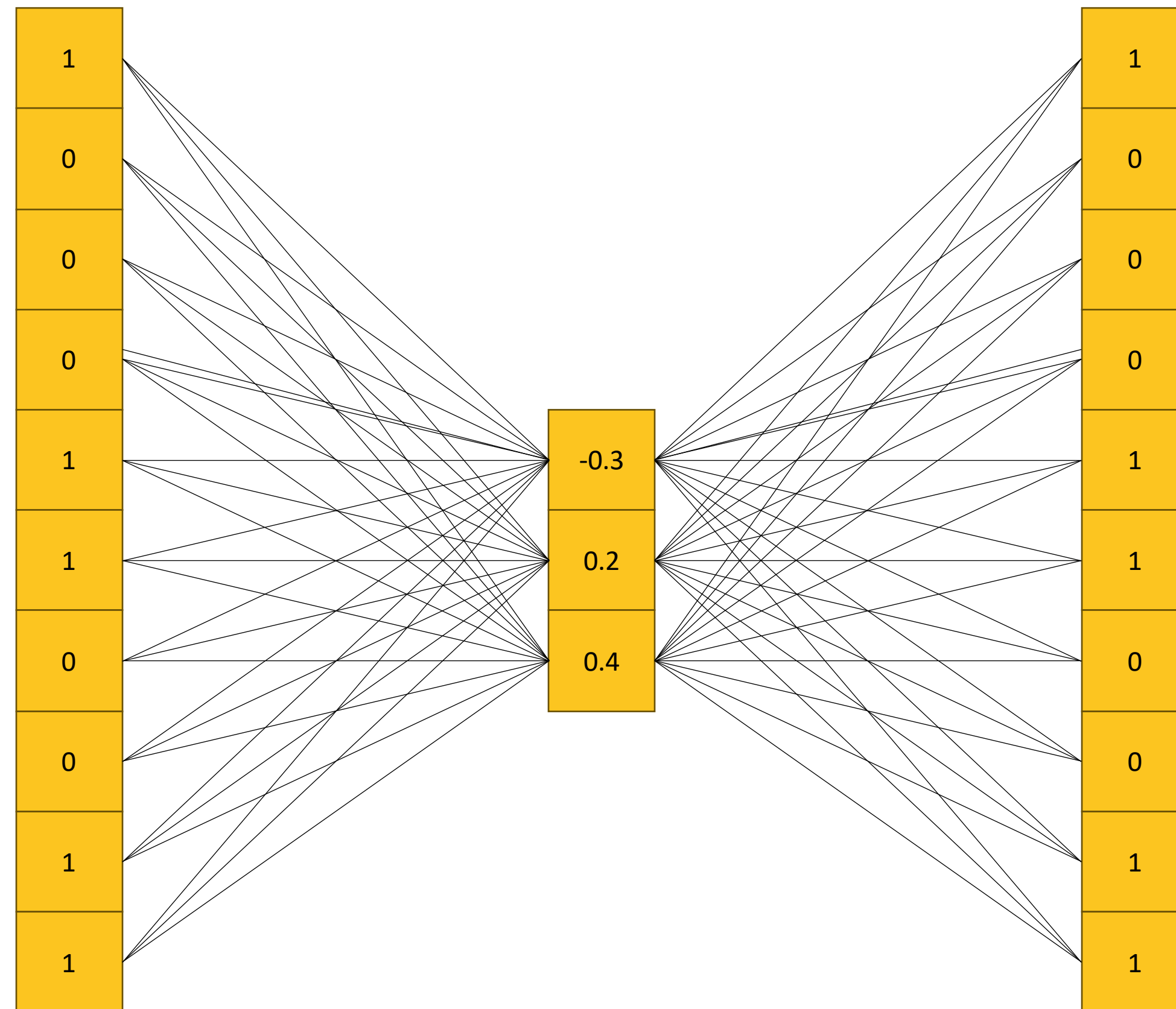
Почему сегодня эта тема?

Пример решения Yandex Cup 2021

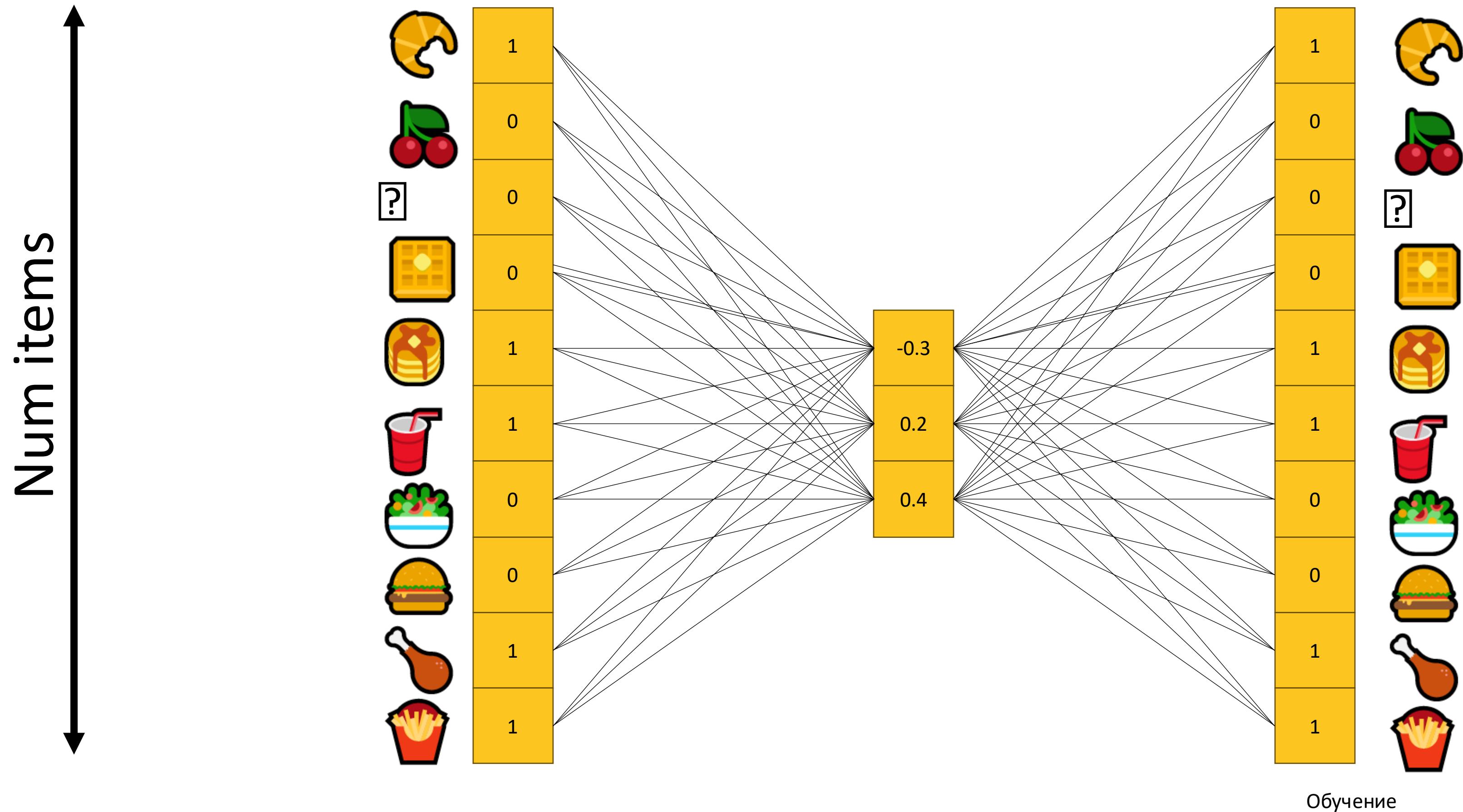


Топ-1 решение на Yandex Cup 2021

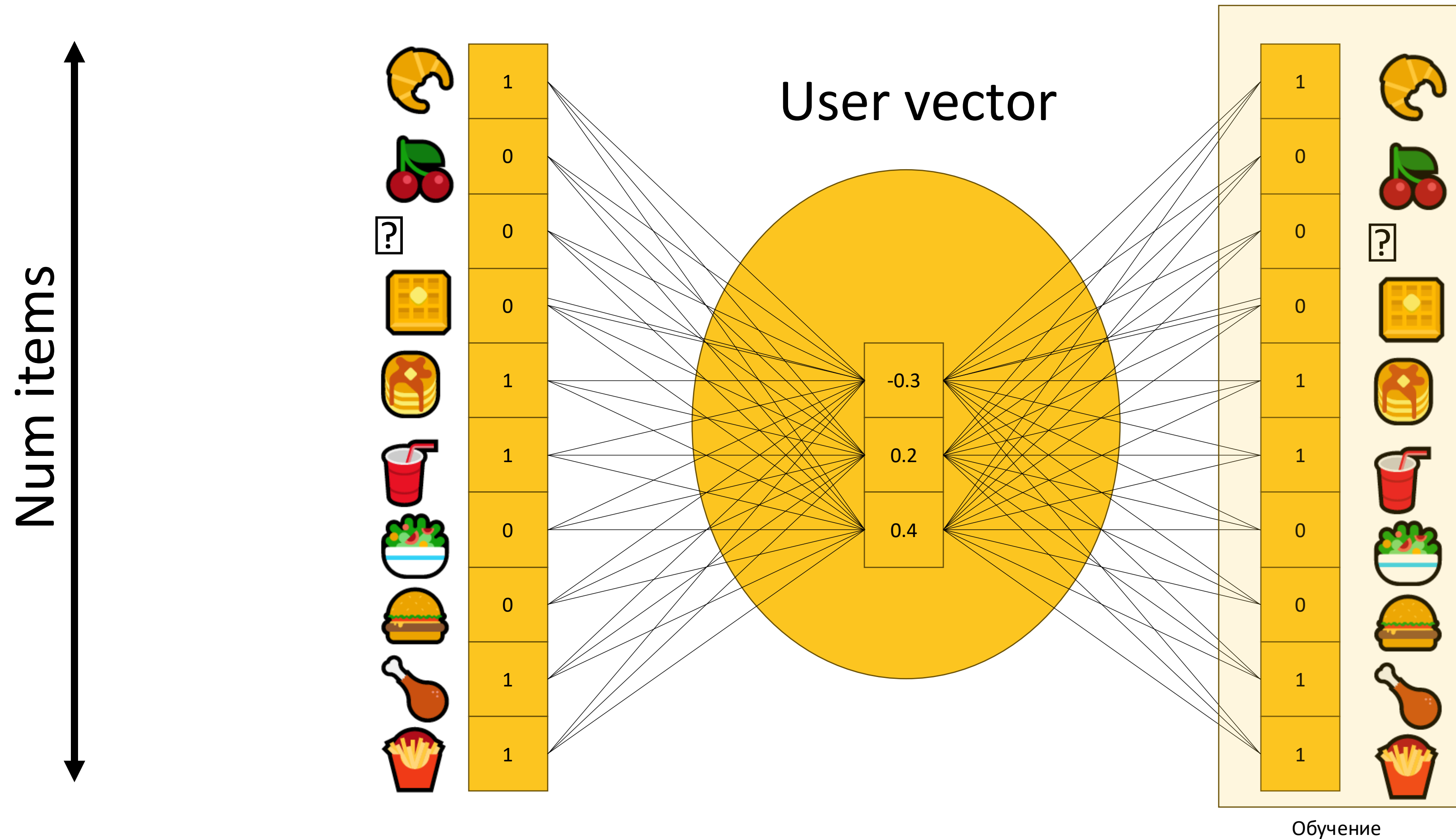
Автоэнкодеры в рекомендациях



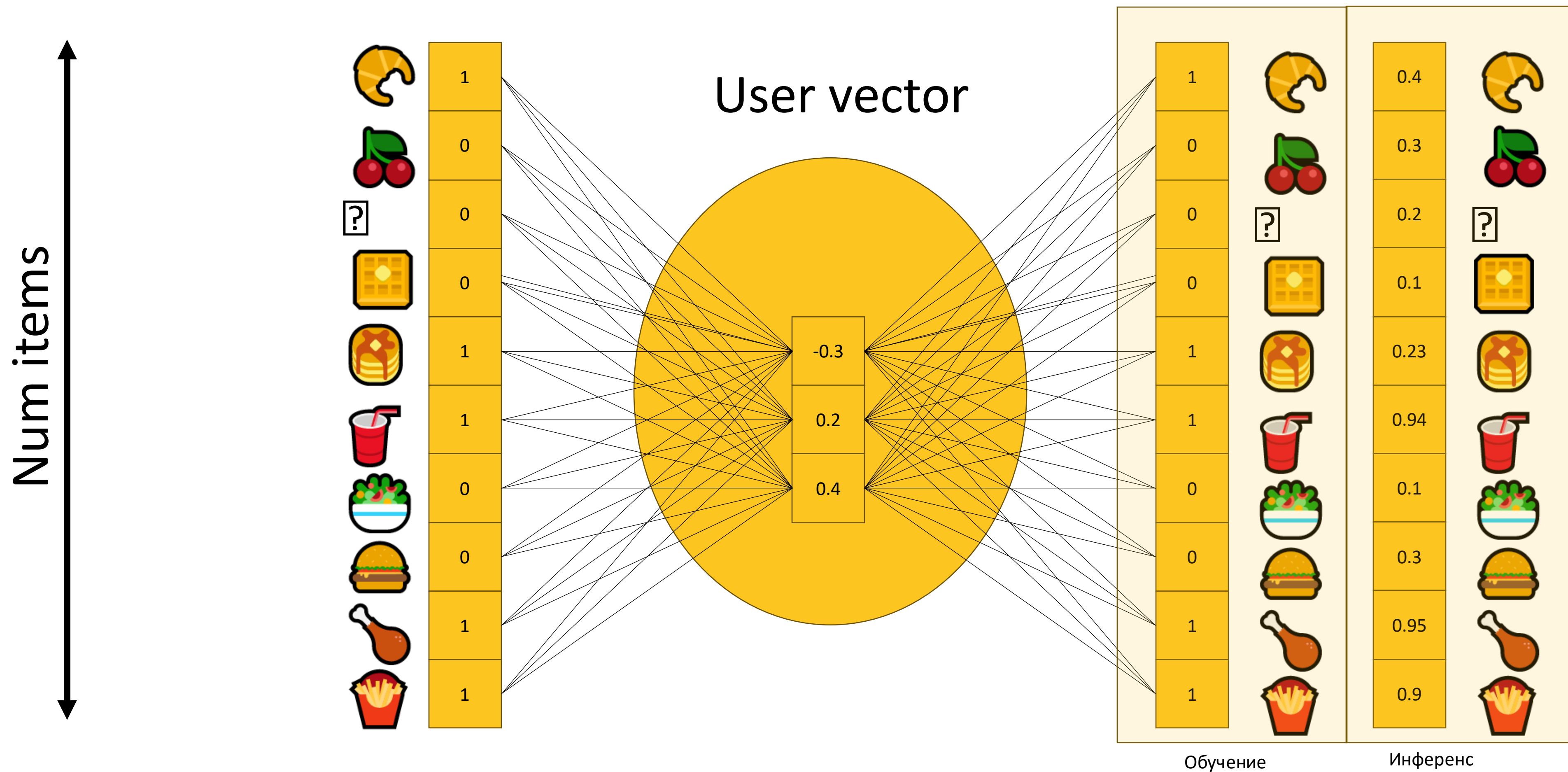
Автоэнкодеры в рекомендациях



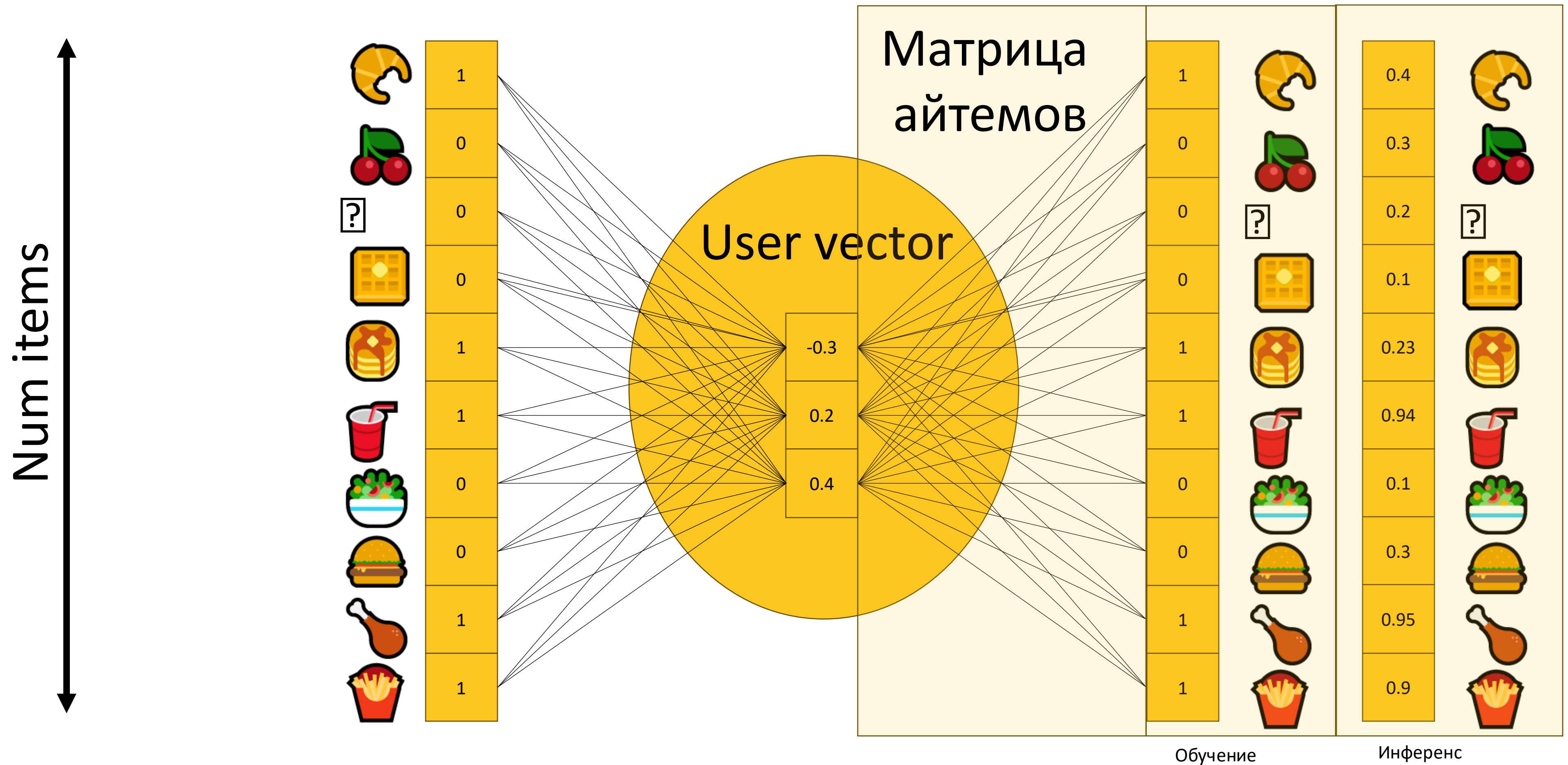
Автоэнкодеры в рекомендациях



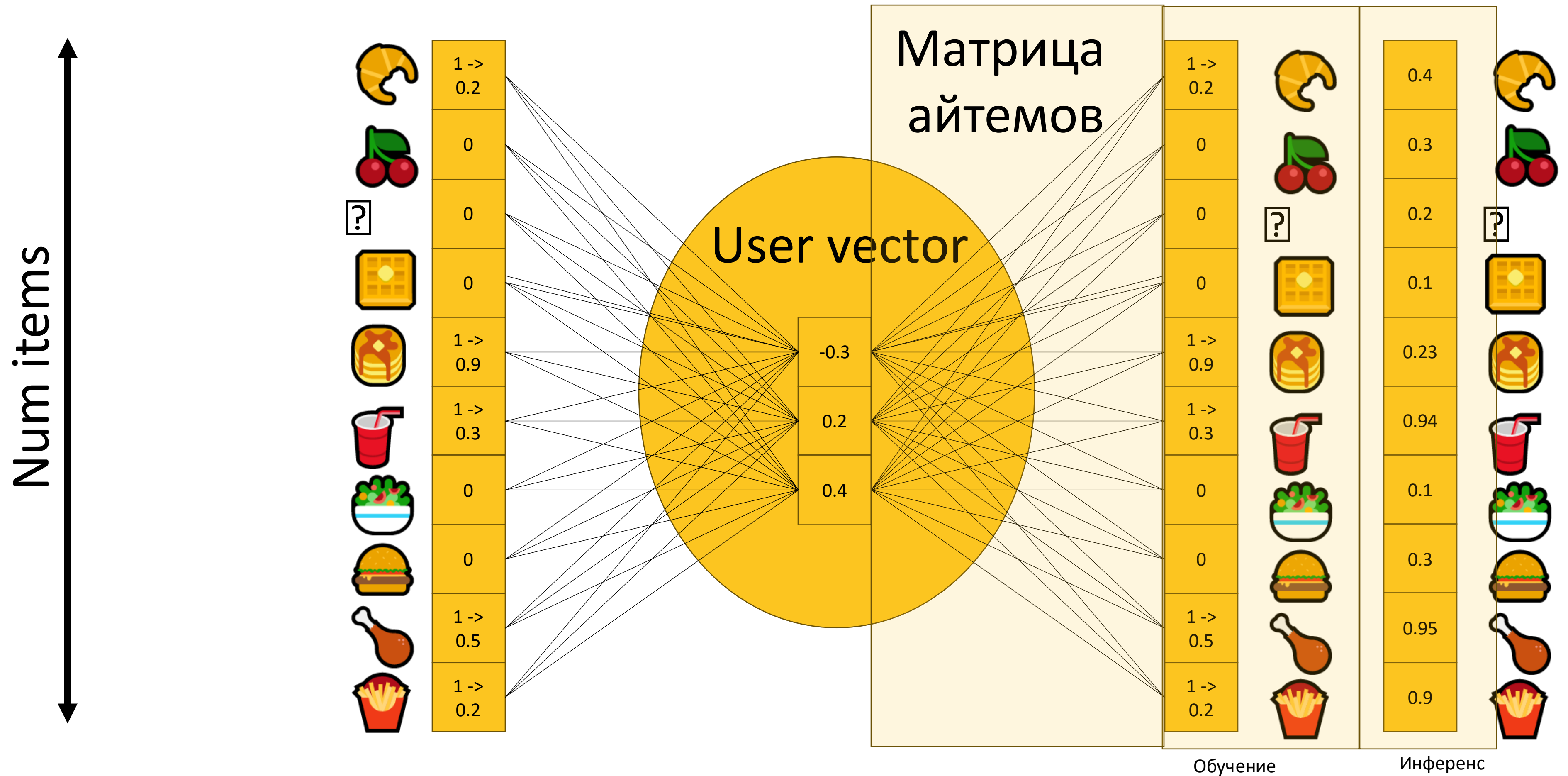
Автоэнкодеры в рекомендациях



Автоэнкодеры в рекомендациях

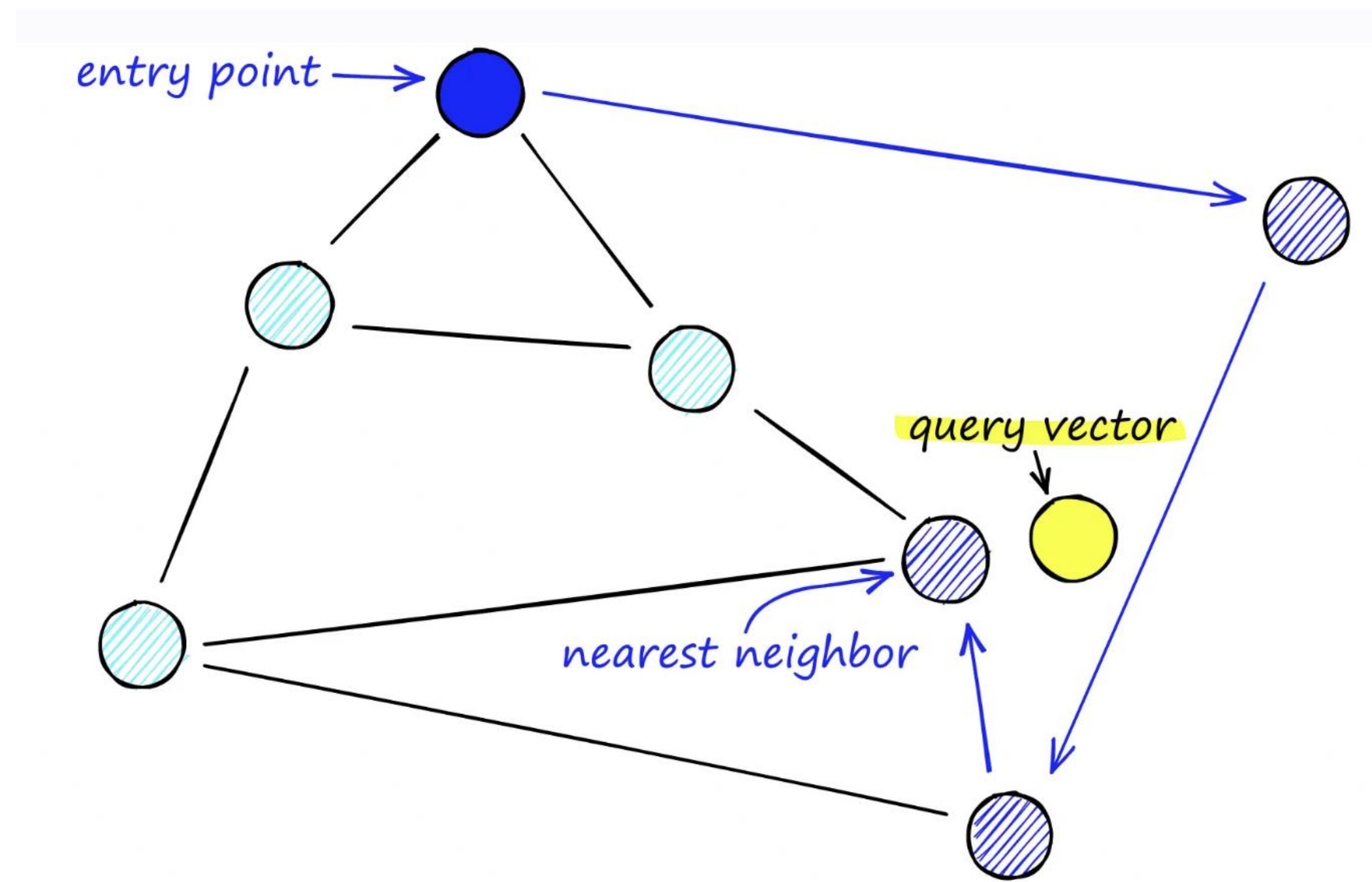


Учет последовательности действий пользователя?

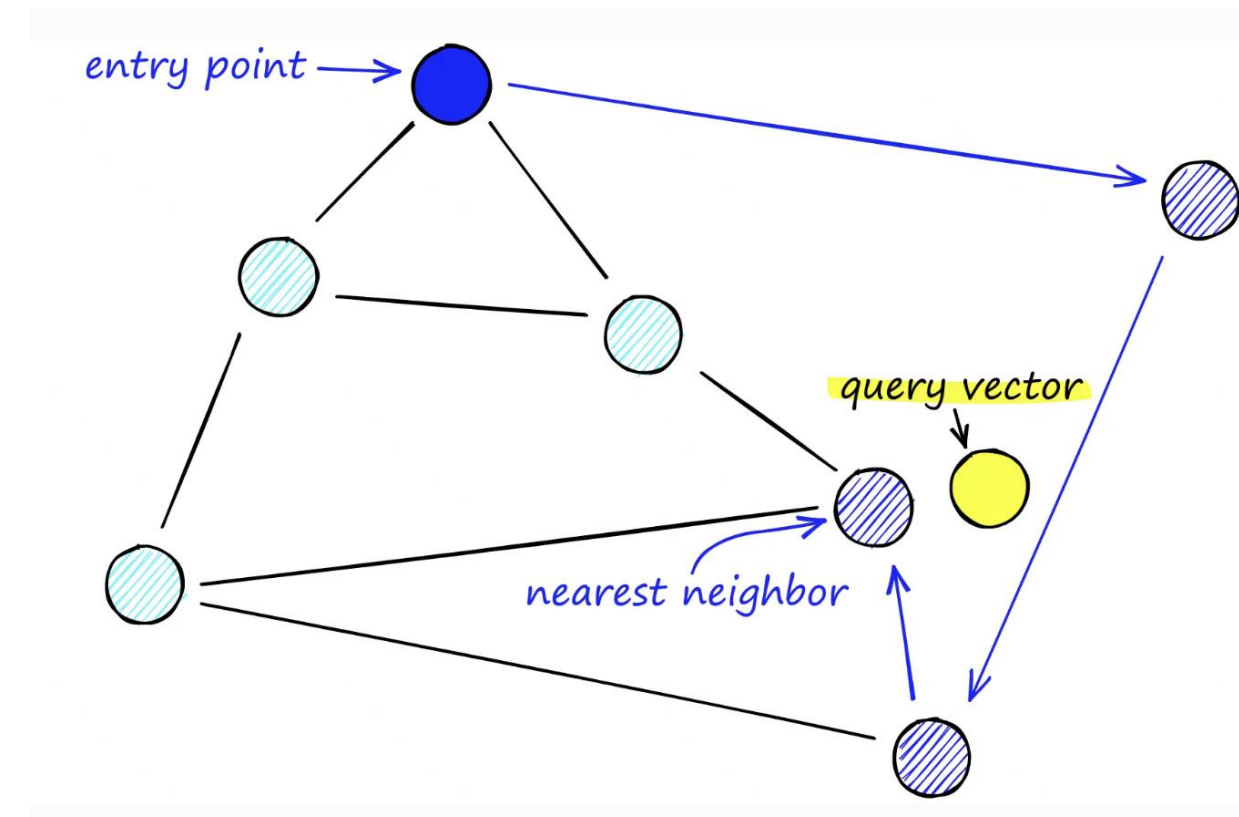
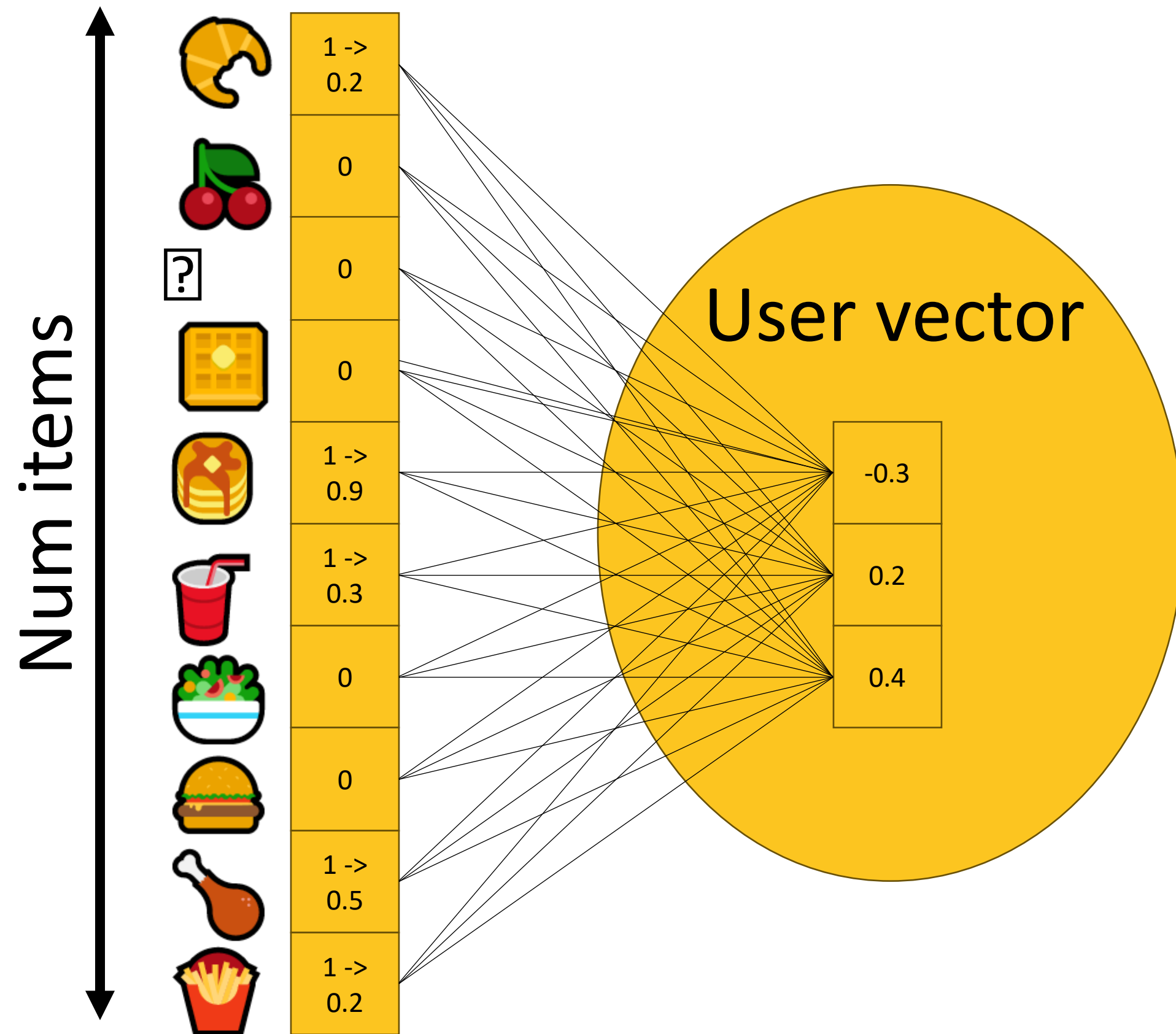


Approximate nearest neighbors (ANN)

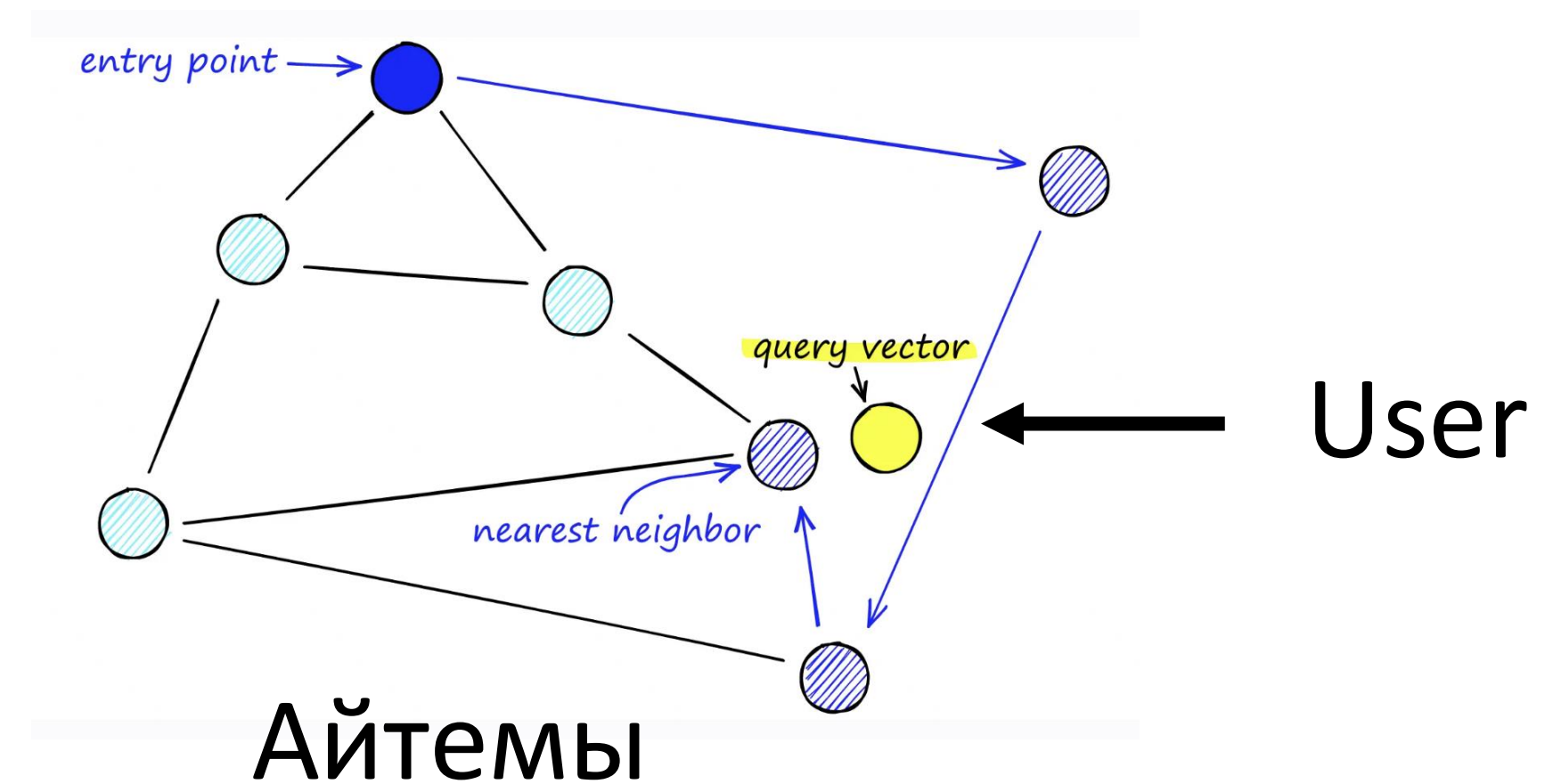
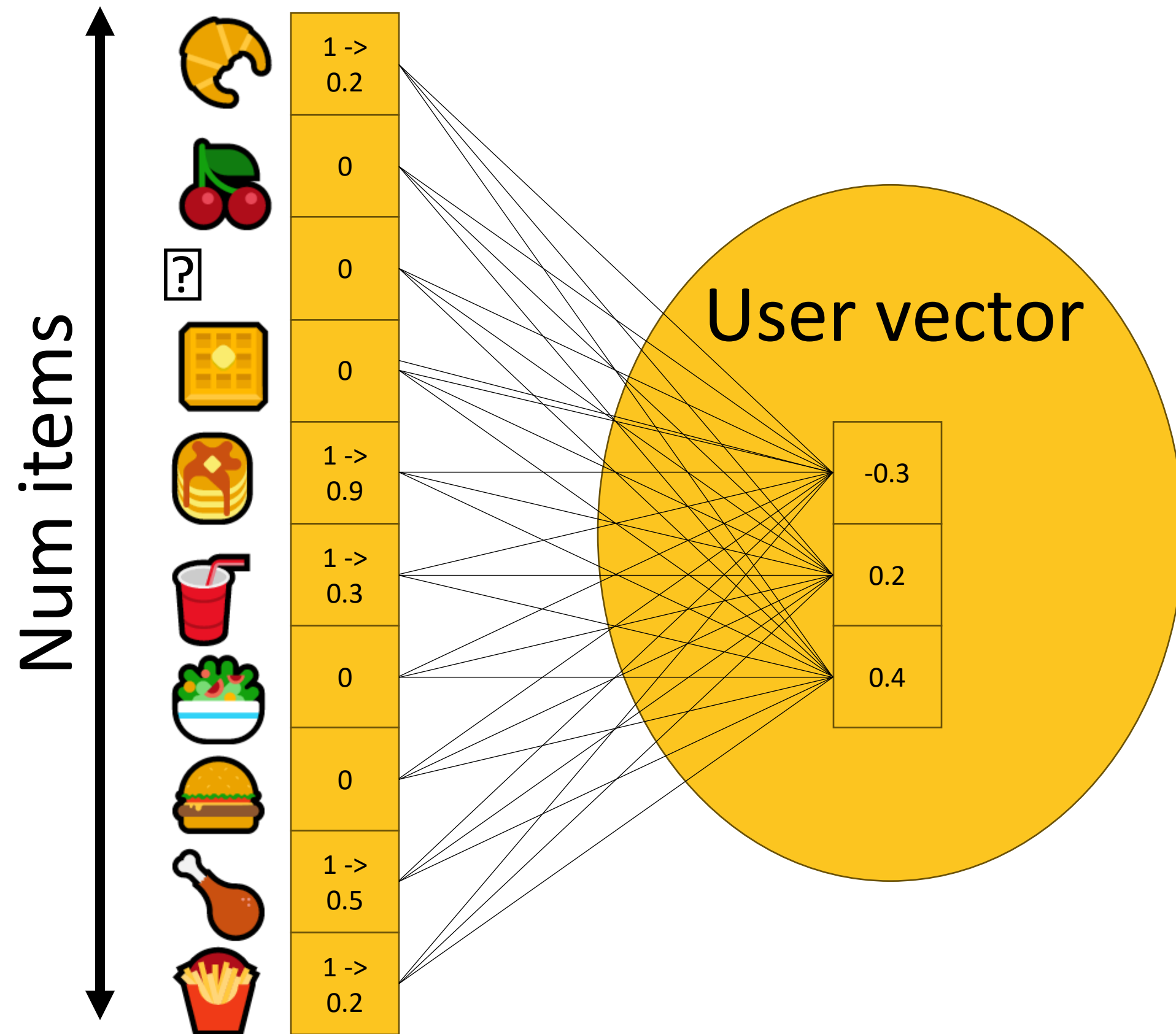
Точка – вектор, надо найти ближайший к query



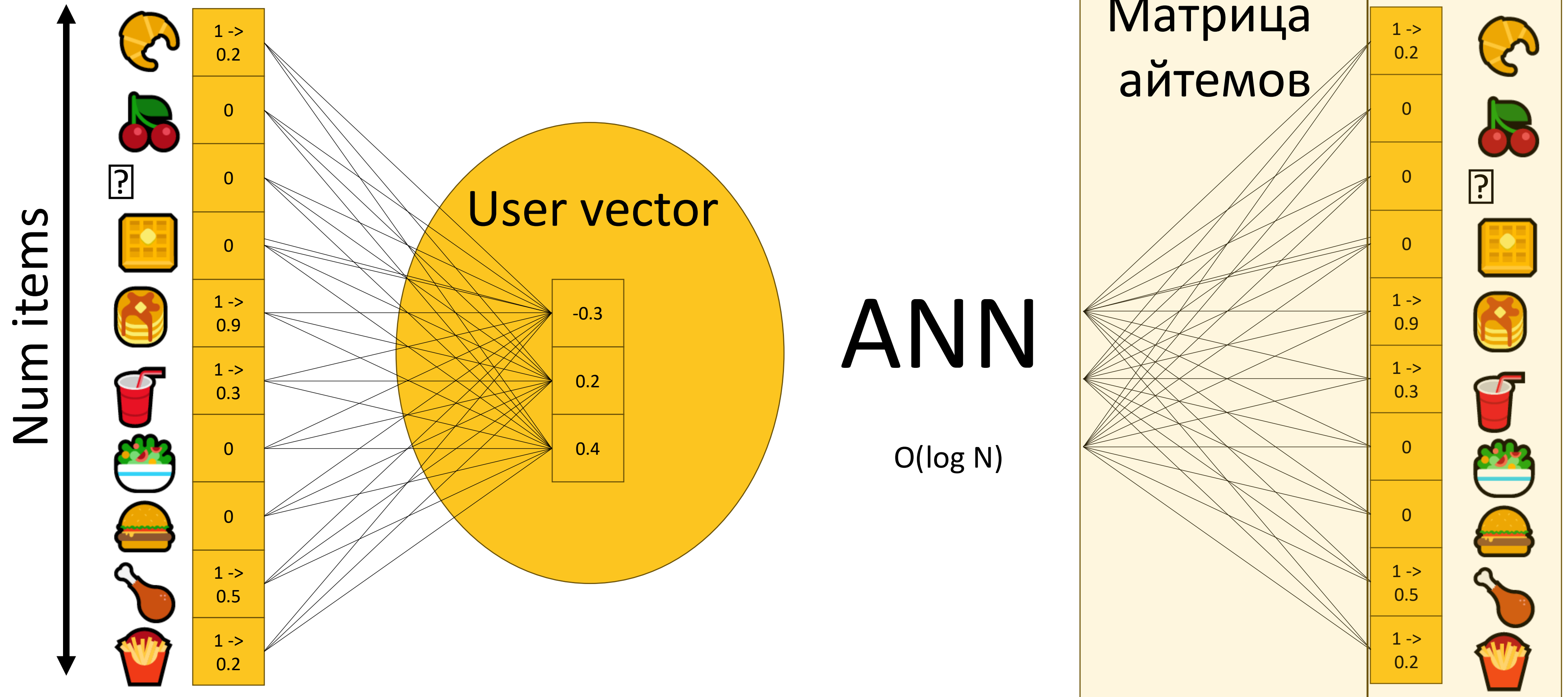
Как применить ANN для рекомендаций?



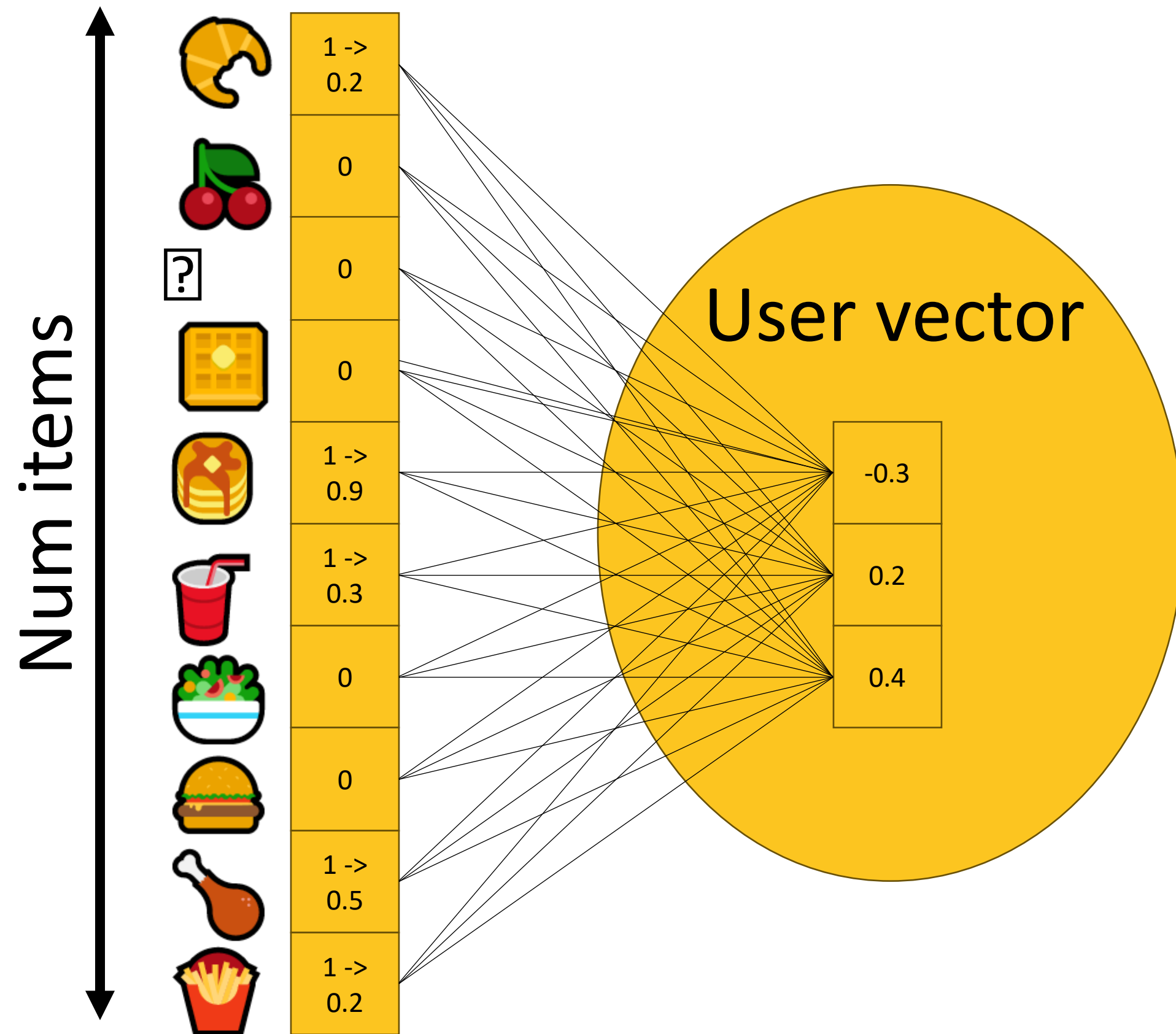
Как применить ANN для рекомендаций?



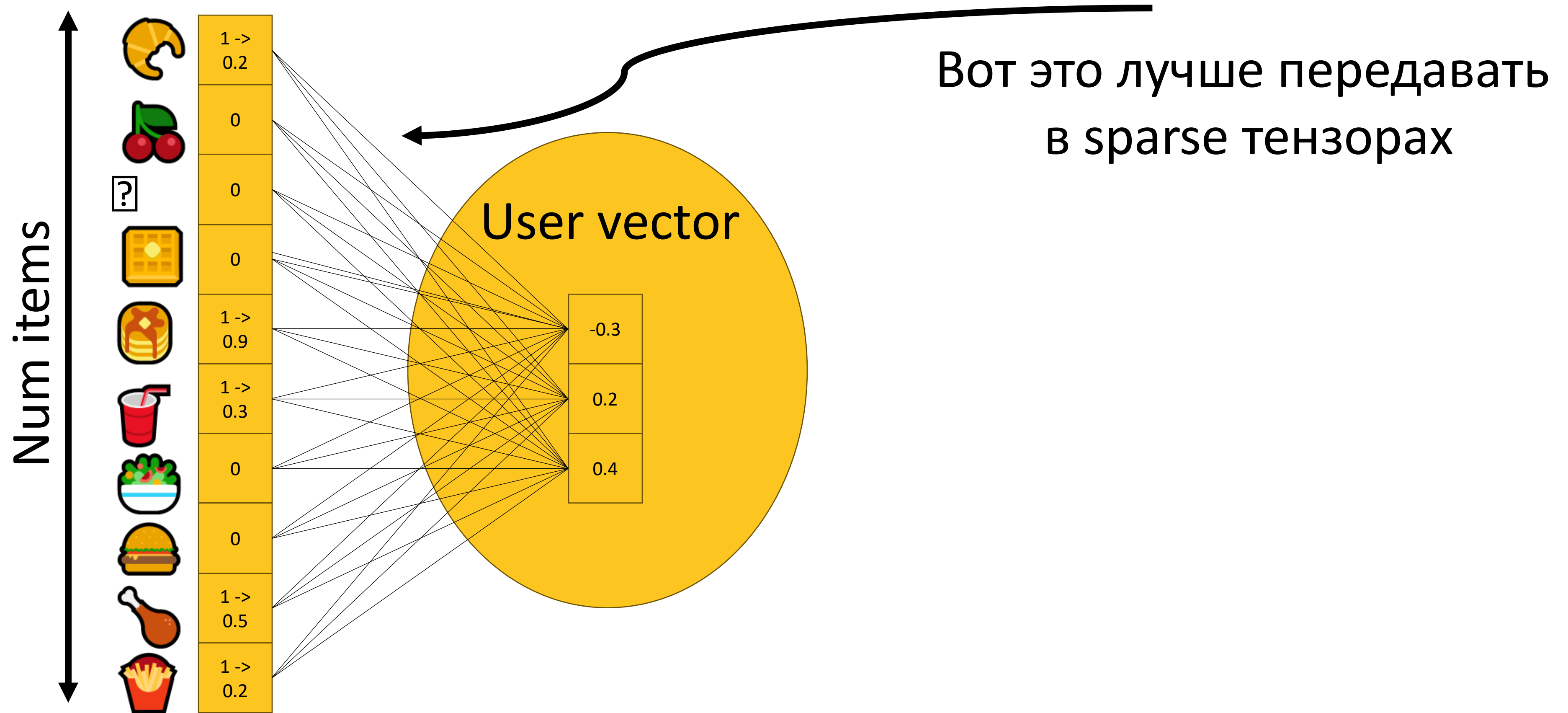
Более быстрый инференс с ANN



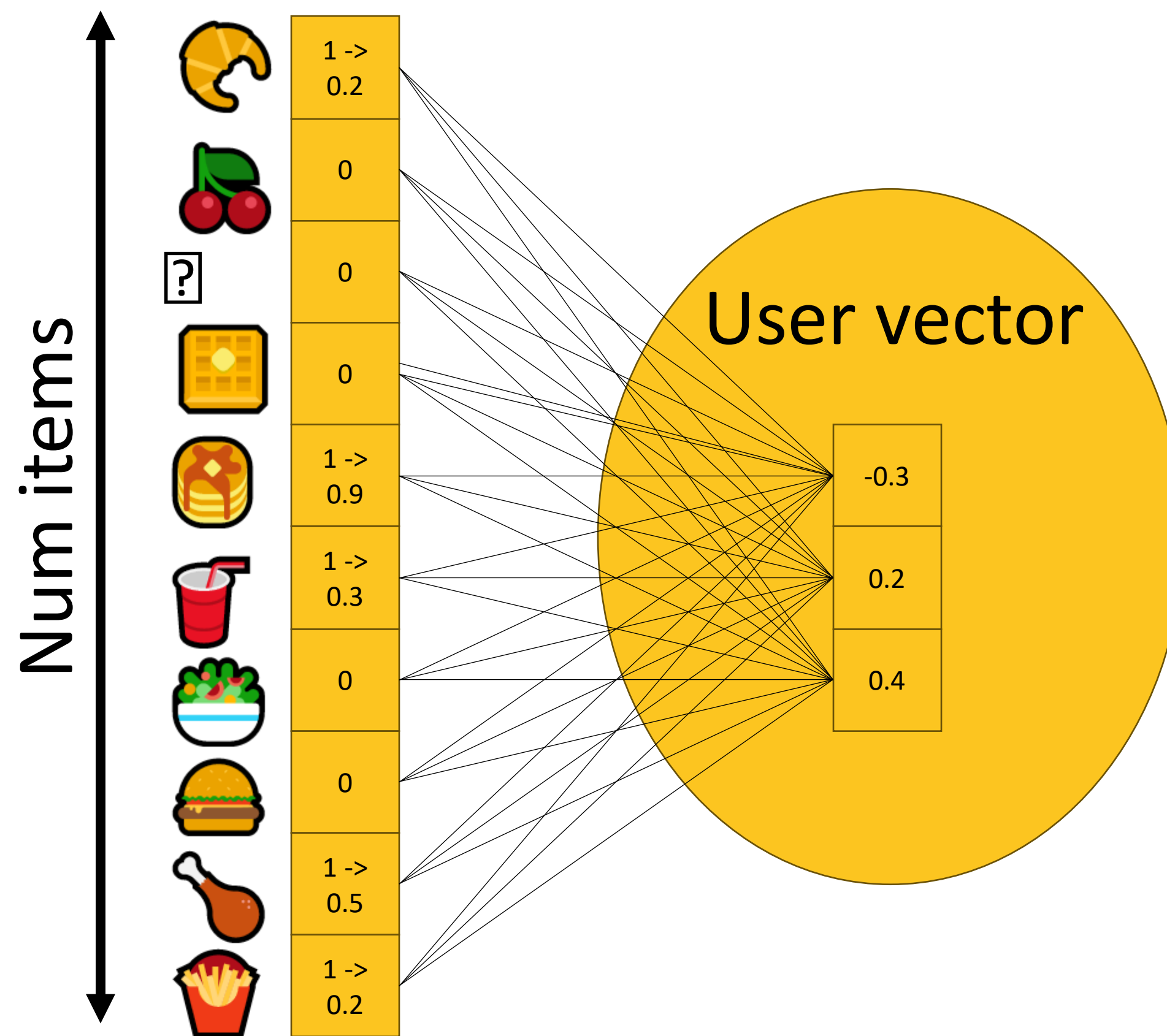
Что можно тут оптимизировать?



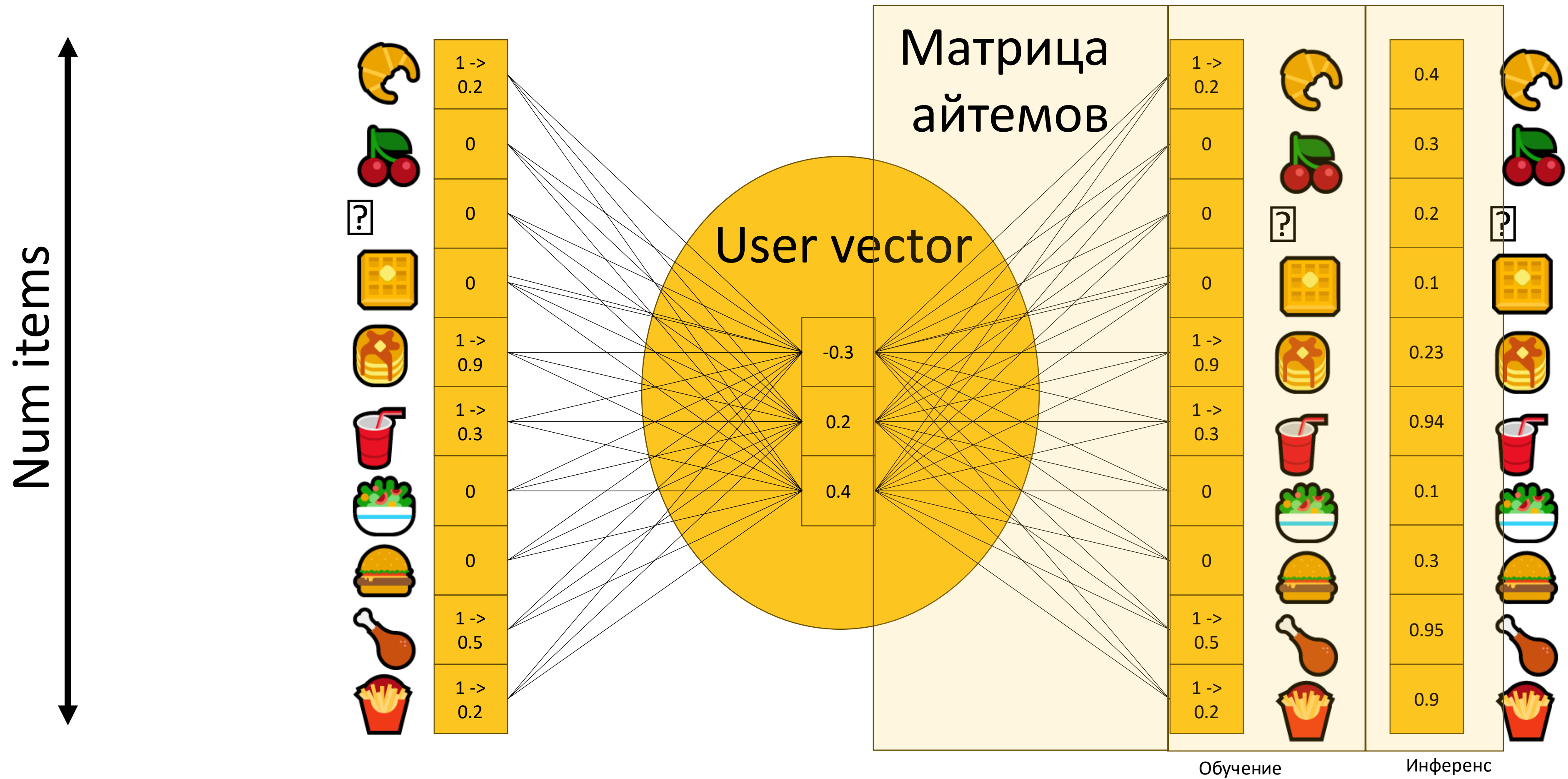
Что можно тут оптимизировать?



Рекомендации по новым юзерам?



Оценка числа параметров модели в N – юзеры, M – айтемы?



Оценка числа параметров модели в N – юзеры, M – айтемы?



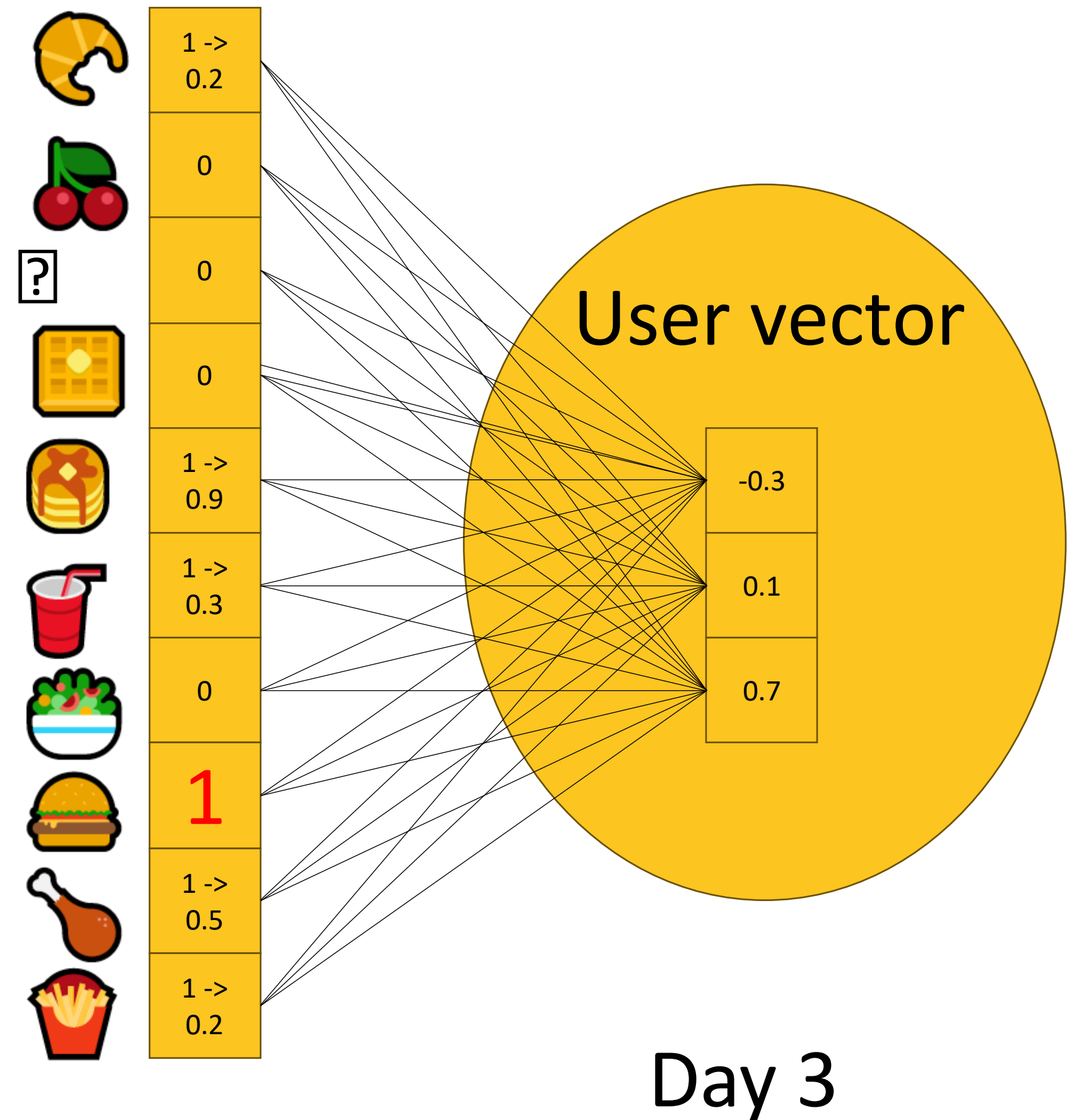
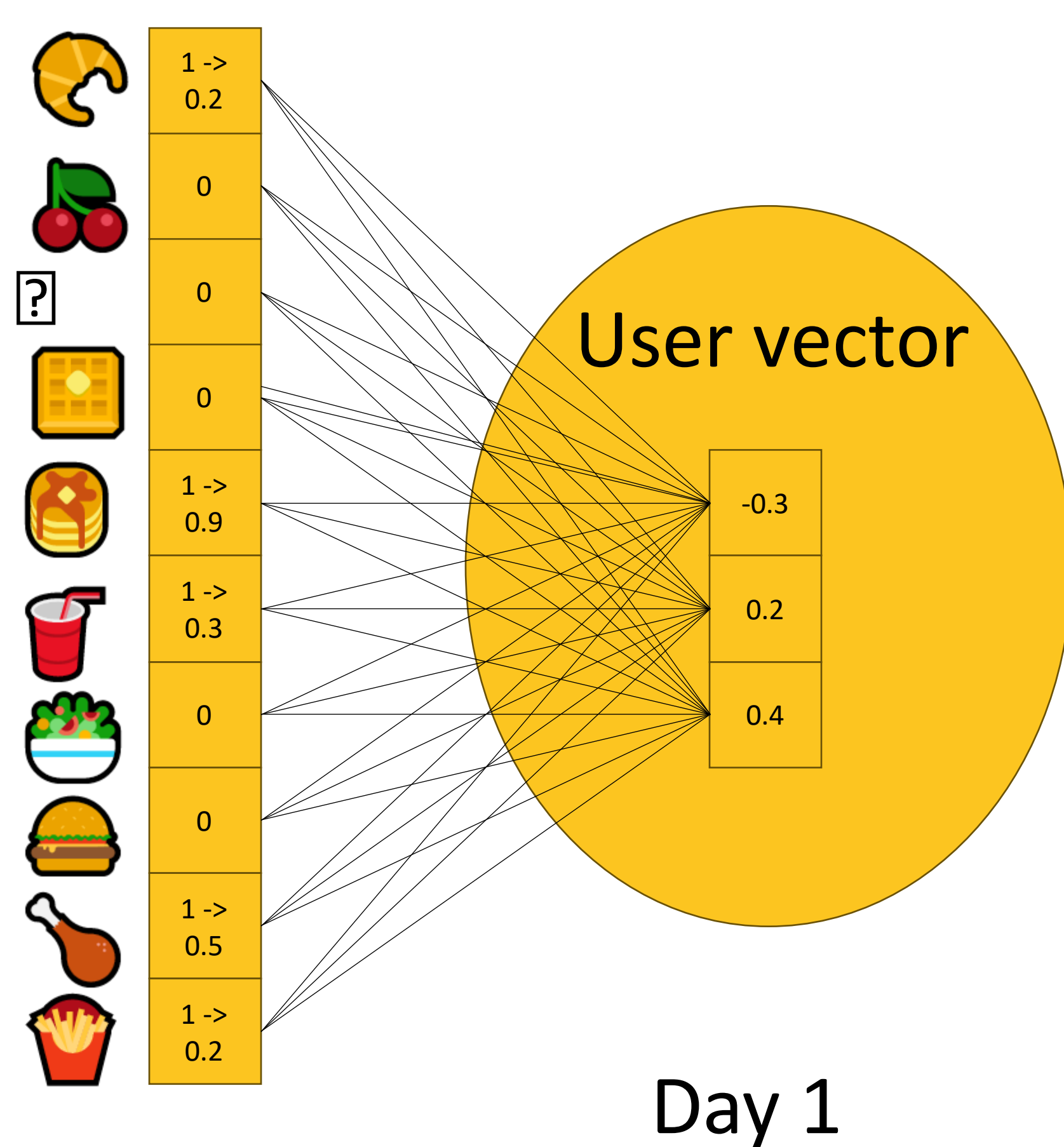
	маленькое	большое
N – число юзеров	Не влияет	Не влияет
M – число айтемов	Не проблема	ANN + может медленнее считаться

Кейс: пусть у вас 20 М юзеров и 3000 айтемов.

Утверждение: VAE можно обучить на 1М людей, а предсказывать на всех 20М.

Проверка: построить график качества от кол-ва юзеров

Веса модели одинаковы – изменятся ли рекомендации?



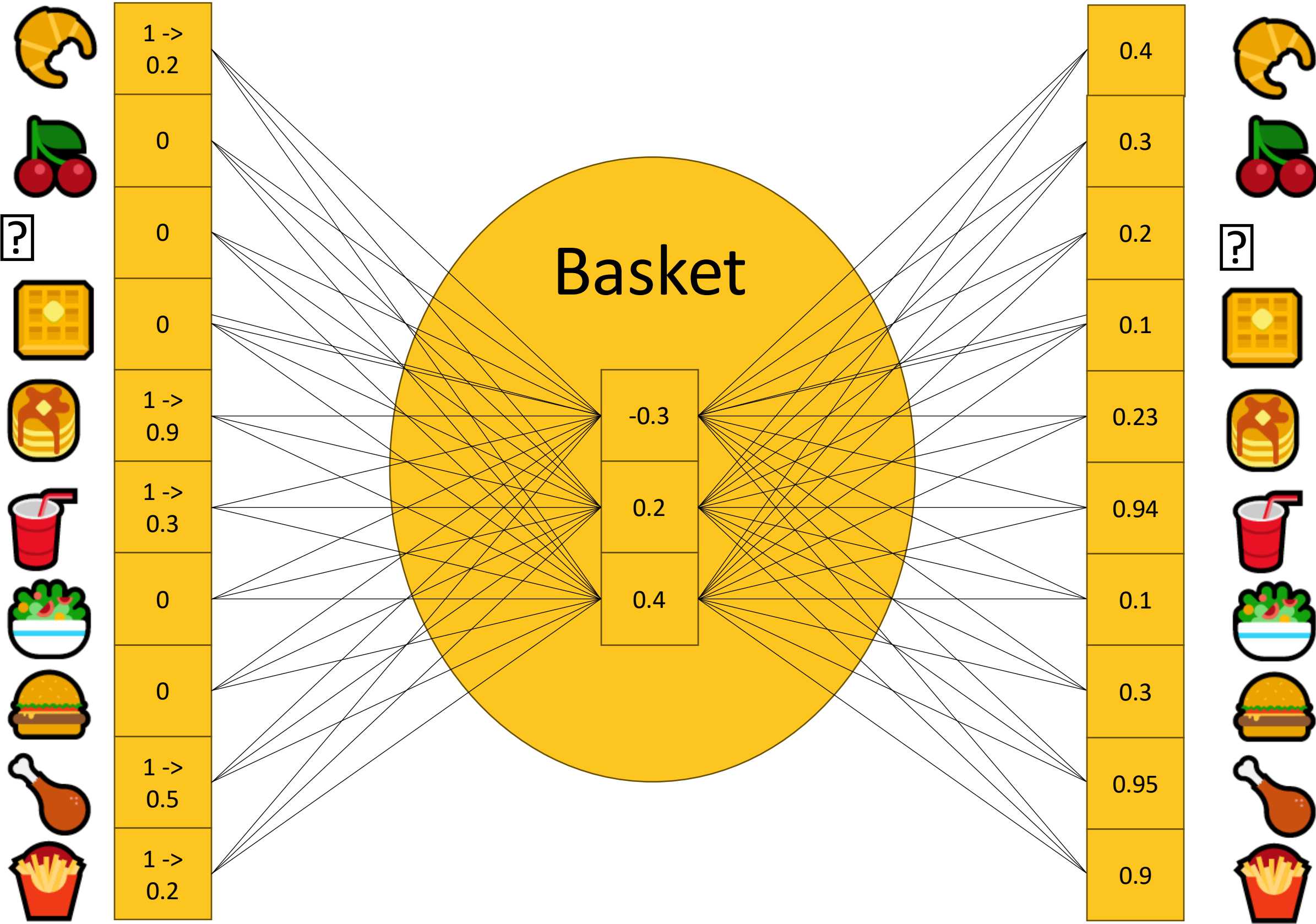
Как часто надо обучать AE-based модель?

	Да	Нет
Новые айтемы появляются часто	Обучение чаще, чтобы иметь возможность их рекомендовать	Можно реже, ведь каталог стабильный, и проблемы холодного старта по айтемам меньше
Новые юзеры появляются часто	Не проблема, можно инференсить их старой моделью	Проблемы нет
Паттерны юзеров в данных быстро меняются	Лучше обучать модель чаще	Можно обучать модель реже
Временной промежуток для обучения (агрегации данных юзера) большой (месяца)	Обучение реже, ведь «через день» пересбор датасета почти ничего не поменяет	Обучение чаще, ведь новый датасет будет действительно новым

Еще кроме рекомендаций применение для бизнеса



Применение для заказов (корзины)



<
Корзина
🔍
🗑️

Палочки ватные Рирі 160шт
40 г

- 1 +

49,99 ₺

Если товара не будет
Позвонить, чтобы заменить

Комментарий для сборщика

Добавить к заказу?

79,99 ₺
Салфетки
влажные
универсальные 80

257 г +

59,99 ₺
Диски ватные Рирі
80шт.

50 г +

29,99 ₺
Салфетки Красная
цена бумажные
24*24см 100шт

110 г +

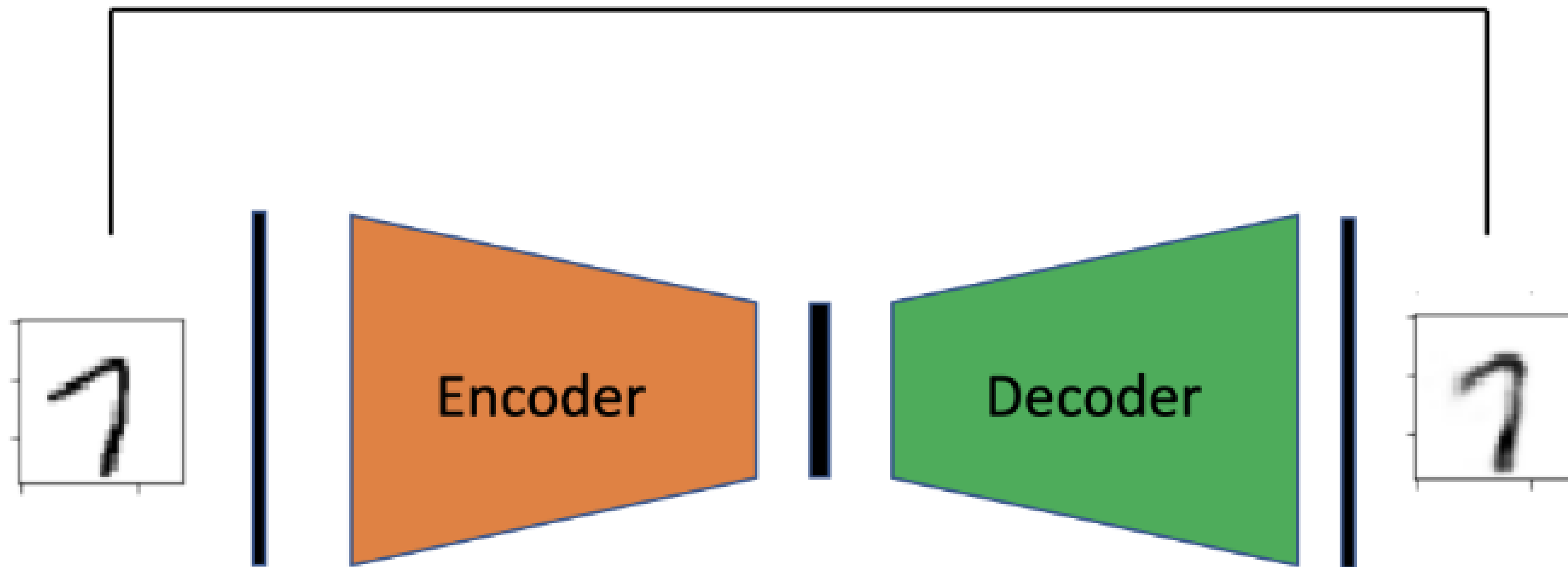
К оформлению
49,99 ₺

Для заказа добавьте еще на 450,01 ₺

Откуда в целом появился автоэнкодер?

Minimize squared error loss:

$$\mathcal{L} = ||\mathbf{x} - Dec(Enc(\mathbf{x}))||_2^2$$



Пример для рекомендаций

Minimize squared error loss:

$$\mathcal{L} = ||\mathbf{x} - Dec(Enc(\mathbf{x}))||_2^2$$

Vector of users
interactions.

1
0
1
1
1
0
⋮
0
0

Encoder

Decoder

Predicted
scores

1.55
0.67
1.13
2.1
0.3
0.1
⋮
1.3
0.7

Variational Autoencoder

1) Minimize squared error loss: (ensures good reconstruction)

$$\mathcal{L}_1 = \|\mathbf{x} - Dec(Enc(\mathbf{x}))\|_2^2 = \sum_{i=1}^d (x_i - x'_i)^2$$

Vector of users
interactions.

1
0
1
1
1
0
...
0
0

$q_w(z|x)$

Encoder

z

$p_w(x|z)$

Decoder

Predicted
scores

1.55
0.67
1.13
2.1
0.3
0.1
...
1.3
0.7

2) Minimize KL
divergence:

(ensures latent space
is continuous and
standard normal
distributed)

$$\mathcal{L}_2 = D_{KL}[N(\mu, \sigma) \| N(0, 1)] = -\frac{1}{2} \sum (1 + \log(\sigma^2) - \mu^2 - \sigma^2)$$

Overall loss: $\mathcal{L} = \alpha \cdot \mathcal{L}_1 + \mathcal{L}_2$

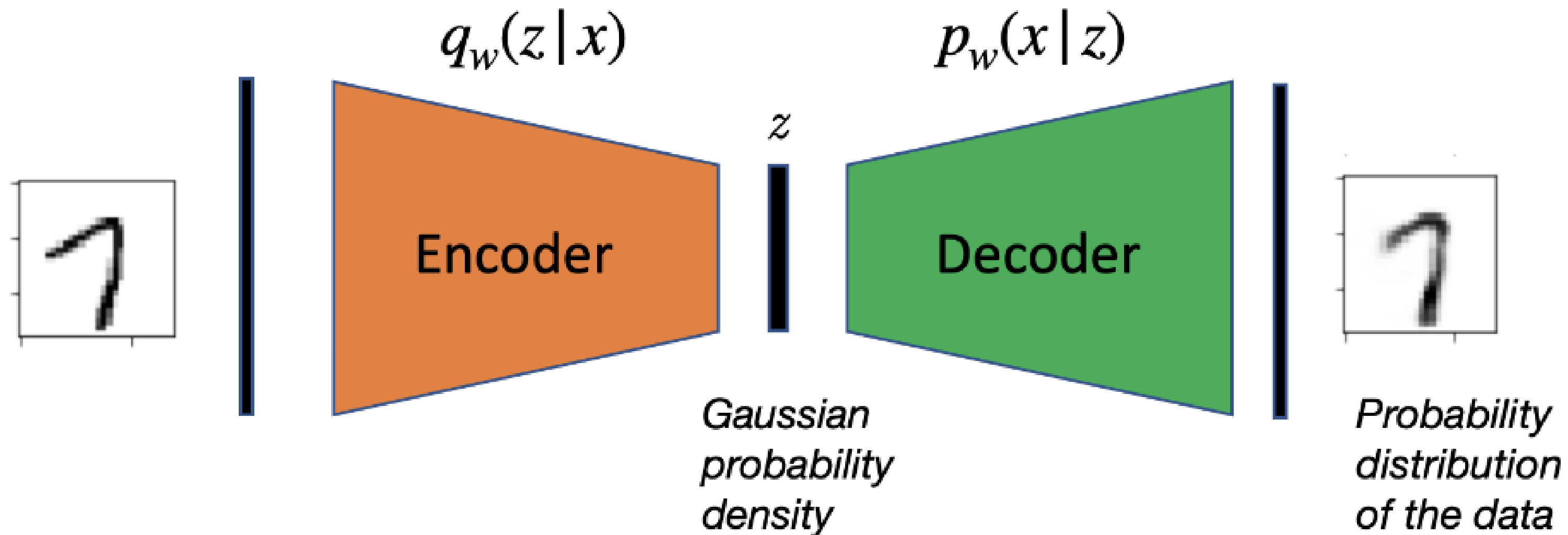
Нелинейные связи!

Variational autoencoder

$$\mathcal{L} = -\mathbb{E}_{z \sim q_w(z|x^{[i]})} [\log p_w(x^{[i]}|z)] + \text{KL} (q_w(z|x^{[i]}) || p(z))$$

Expected neg. log likelihood
term; wrt to encoder distribution

Kullback-Leibler divergence term
where $p(z) = \mathcal{N}(\mu = 0, \sigma^2 = 1)$



Виды автоэнкодеров

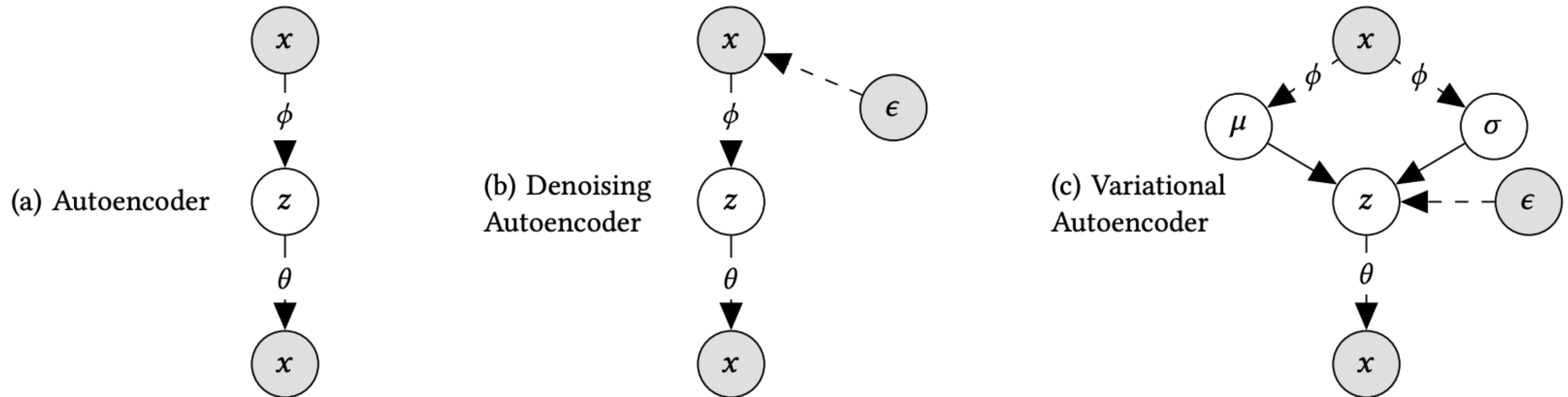


Figure 2: A taxonomy of autoencoders. The dotted arrows denote a sampling operation.

EASE

CDAE/MultiDAE

MultiVAE

Адаптація для рекомендацій

Vector of users interactions.

1
0
1
1
1
0
⋮
⋮
0
0

Encoder

mean vector

2.23
-0.5
1.11
0.76

Latent representation of user

1.1
-1
1.1
0.5

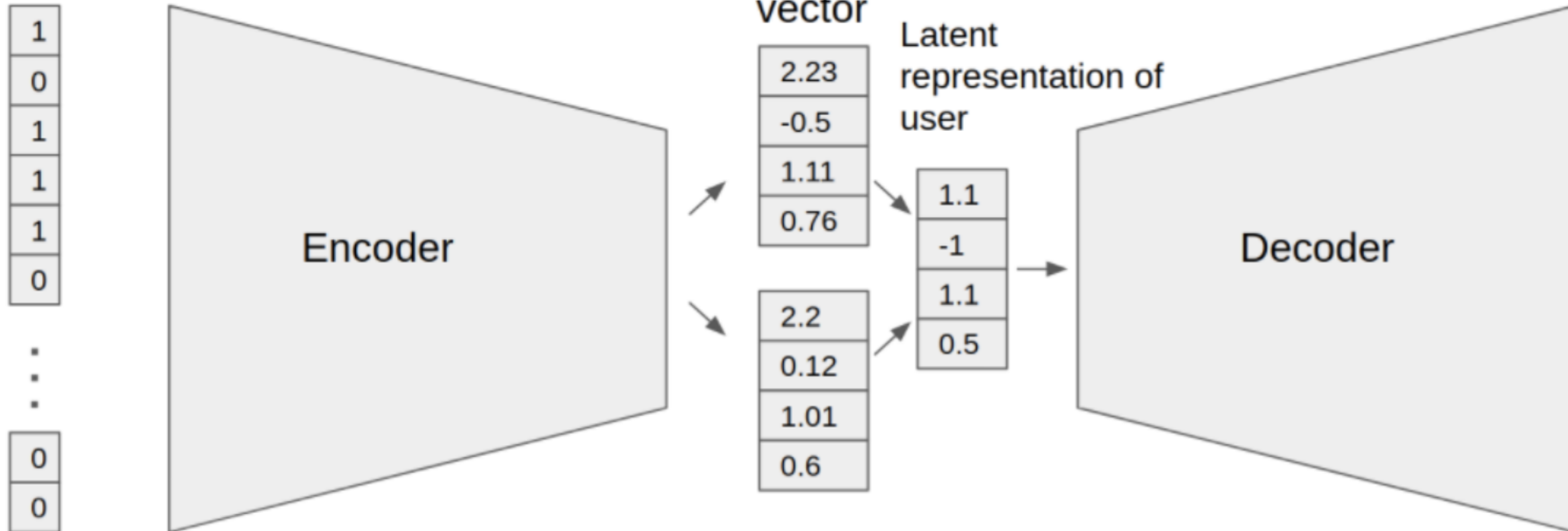
2.2
0.12
1.01
0.6

Standard deviation vector

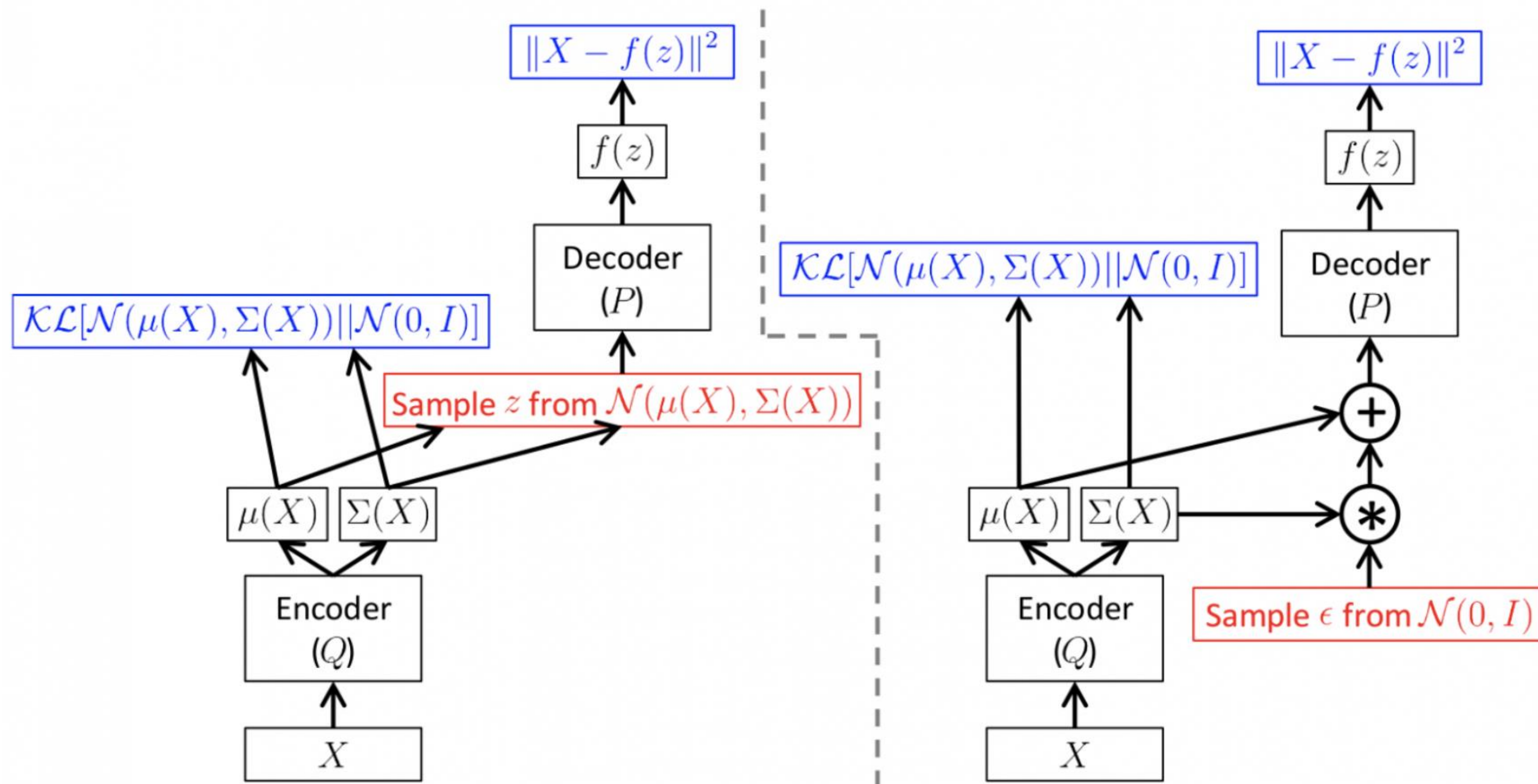
Decoder

Predicted scores

1.55
0.67
1.13
2.1
0.3
0.1
⋮
⋮
1.3
0.7



Reparametrization trick



Multi-VAE



arXiv:1802.05814v1 [stat.ML] 16 Feb 2018

Variational Autoencoders for Collaborative Filtering

Dawen Liang
Netflix
Los Gatos, CA
dliang@netflix.com

Matthew D. Hoffman
Google AI
San Francisco, CA
mhoffman@google.com

Rahul G. Krishnan
MIT
Cambridge, MA
rahulgk@mit.edu

Tony Jebara
Netflix
Los Gatos, CA
tjebara@netflix.com

ABSTRACT

We extend variational autoencoders (VAEs) to collaborative filtering for implicit feedback. This non-linear probabilistic model enables us to go beyond the limited modeling capacity of linear factor models which still largely dominate collaborative filtering research. We introduce a generative model with multinomial likelihood and use Bayesian inference for parameter estimation. Despite widespread use in language modeling and economics, the multinomial likelihood receives less attention in the recommender systems literature. We introduce a different regularization parameter for the learning objective, which proves to be crucial for achieving competitive performance. Remarkably, there is an efficient way to tune the parameter using annealing. The resulting model and learning algorithm has information-theoretic connections to maximum entropy discrimination and the information bottleneck principle. Empirically, we show that the proposed approach significantly outperforms several state-of-the-art baselines, including two recently-proposed neural network approaches, on several real-world datasets. We also provide extended experiments comparing the multinomial likelihood with other commonly used likelihood functions in the latent factor collaborative filtering literature and show favorable results. Finally, we identify the pros and cons of employing a principled Bayesian inference approach and characterize settings where it provides the most significant improvements.

KEYWORDS

Recommender systems, collaborative filtering, implicit feedback, variational autoencoder, Bayesian models

ACM Reference Format:

Dawen Liang, Rahul G. Krishnan, Matthew D. Hoffman, and Tony Jebara. 2018. Variational Autoencoders for Collaborative Filtering. In *Proceedings of The 2018 Web Conference (WWW 2018)*. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3178876.3186150>

This paper is published under the Creative Commons Attribution-NonCommercial-NoDerivs 4.0 International (CC BY-NC-ND 4.0) license. Authors reserve their rights to disseminate the work on their personal and corporate Web sites with the appropriate attribution.

WWW 2018, April 23–27, 2018, Lyon, France

© 2018 IW3C2 (International World Wide Web Conference Committee), published under Creative Commons CC BY-NC-ND 4.0 License.

ACM ISBN 978-1-4503-5639-8/18/04.

<https://doi.org/10.1145/3178876.3186150>

1 INTRODUCTION

Recommender systems are an integral component of the web. In a typical recommendation system, we observe how a set of users interacts with a set of items. Using this data, we seek to show users a set of previously unseen items they will like. As the web grows in size, good recommendation systems will play an important part in helping users interact more effectively with larger amounts of content.

Collaborative filtering is among the most widely applied approaches in recommender systems. Collaborative filtering predicts what items a user will prefer by discovering and exploiting the similarity patterns across users and items. Latent factor models [13, 19, 38] still largely dominate the collaborative filtering research literature due to their simplicity and effectiveness. However, these models are inherently linear, which limits their modeling capacity. Previous work [27] has demonstrated that adding carefully crafted non-linear features into the linear latent factor models can significantly boost recommendation performance. Recently, a growing body of work involves applying neural networks to the collaborative filtering setting with promising results [14, 41, 51, 54].

Here, we extend variational autoencoders (VAEs) [24, 37] to collaborative filtering for implicit feedback. VAEs generalize linear latent-factor models and enable us to explore non-linear probabilistic latent-variable models, powered by neural networks, on large-scale recommendation datasets. We propose a neural generative model with multinomial conditional likelihood. Despite being widely used in language modeling and economics [5, 30], multinomial likelihoods appear less studied in the collaborative filtering literature, particularly within the context of latent-factor models. Recommender systems are often evaluated using ranking-based measures, such as mean average precision and normalized discounted cumulative gain [21]. Top- N ranking loss is difficult to optimize directly and previous work on direct ranking loss minimization resorts to relaxations and approximations [49, 50]. Here, we show that the multinomial likelihoods are well-suited for modeling implicit feedback data, and are a closer proxy to the ranking loss relative to more popular likelihood functions such as Gaussian and logistic.

Though recommendation is often considered a big-data problem (due to the huge numbers of users and items typically present in a recommender system), we argue that, in contrast, it represents a uniquely challenging “small-data” problem: most users only interact with a tiny proportion of the items and our goal is to collectively

Multi-VAE



$u \in \{1, \dots, U\}$ - пользователи,

$i \in \{1, \dots, I\}$ - объекты (айтемы),

$\mathbf{X} \in \mathbb{N}^{U \times I}$ - матрица взаимодействий из кликов в бинарном виде.

$$\mathbf{x}_u = [X_{u1}, \dots, X_{uI}]^T \in \mathbb{N}^I$$

- вектор из мешка слов по айтемам с числом кликов пользователя по каждому из них.

$$\mathbf{z}_u \sim \mathcal{N}(0, \mathbf{I}_K), \quad \pi(\mathbf{z}_u) \propto \exp\{f_\theta(\mathbf{z}_u)\},$$

$$\mathbf{x}_u \sim \text{Mult}(N_u, \pi(\mathbf{z}_u)).$$

Multi-VAE



$$\log p_{\theta}(\mathbf{x}_u | \mathbf{z}_u) \stackrel{c}{=} \sum_i x_{ui} \log \pi_i(\mathbf{z}_u).$$

$$\log p_{\theta}(\mathbf{x}_u | \mathbf{z}_u) \stackrel{c}{=} - \sum_i \frac{c_{ui}}{2} (x_{ui} - f_{ui})^2.$$

$$\log p_{\theta}(\mathbf{x}_u | \mathbf{z}_u) = \sum_i x_{ui} \log \sigma(f_{ui}) + (1 - x_{ui}) \log(1 - \sigma(f_{ui})),$$

$$\log p_{\theta}(\mathbf{x}_u | \mathbf{z}_u) \stackrel{c}{=} \sum_i x_{ui} \log \pi_i(\mathbf{z}_u).$$

$$\mathbf{x}_u = [X_{u1}, \dots, X_{uI}]^T \in \mathbb{N}^I$$

$$\log p_{\theta}(\mathbf{x}_u | \mathbf{z}_u) \stackrel{c}{=} - \sum_i \frac{c_{ui}}{2} (x_{ui} - f_{ui})^2.$$

$$\log p_{\theta}(\mathbf{x}_u | \mathbf{z}_u) = \sum_i x_{ui} \log \sigma(f_{ui}) + (1 - x_{ui}) \log(1 - \sigma(f_{ui})),$$

Table 4: Comparison of Mult-VAE^{PR} and Mult-DAE with different likelihood functions at the output layer on ML-20M. The standard error is around 0.002 (the results on the other two datasets are similar.) The multinomial likelihood performs better than the other two commonly-used likelihoods from the collaborative filtering literature.

	Recall@20	Recall@50	NDCG@100
Mult-VAE ^{PR}	0.395	0.537	0.426
Gaussian-VAE ^{PR}	0.383	0.523	0.415
Logistic-VAE ^{PR}	0.388	0.523	0.419
Mult-DAE	0.387	0.524	0.419
Gaussian-DAE	0.376	0.515	0.409
Logistic-DAE	0.381	0.516	0.414

Embarrassingly Shallow Autoencoders for Sparse Data*

Harald Steck
Netflix
Los Gatos, California
hsteck@netflix.com

The predicted score $S_{u,j}$ for an item $j \in \mathcal{I}$ given a user $u \in \mathcal{U}$ is defined by the dot product

$$S_{uj} = X_{u,\cdot} \cdot B_{\cdot,j}, \quad (1)$$

where $X_{u,\cdot}$ refers to row u , and $B_{\cdot,j}$ to column j .

$$\begin{aligned} \min_B \quad & ||X - XB||_F^2 + \lambda \cdot ||B||_F^2 \\ \text{s.t.} \quad & \text{diag}(B) = 0 \end{aligned}$$

EASE имеет решение в явном виде

Embarrassingly Shallow Autoencoders for Sparse Data*

Harald Steck
Netflix
Los Gatos, California
hsteck@netflix.com

The predicted score $S_{u,j}$ for an item $j \in \mathcal{I}$ given a user $u \in \mathcal{U}$ is defined by the dot product

$$S_{uj} = X_{u,\cdot} \cdot B_{\cdot,j}, \quad (1)$$

where $X_{u,\cdot}$ refers to row u , and $B_{\cdot,j}$ to column j .

$$\hat{P} \triangleq (X^\top X + \lambda I)^{-1},$$

$$\hat{B}_{i,j} = \begin{cases} 0 & \text{if } i = j \\ -\frac{\hat{P}_{ij}}{\hat{P}_{jj}} & \text{otherwise.} \end{cases}$$

Доказательство
можете почитать в
статье

Преимущества EASE

- 1) Имеет решение в явном виде, не требует обучения по эпохам
- 2) Явное решение оптимально в смысле поставленной задачи
- 3) Можно один раз обучить матрицу B , и использовать ее на разных юзерах
- 4) Имеет всего один гиперпараметр
- 5) Устойчив к варьированию гиперпараметра (в отличие от SLIM). В целом 100 – хорошее значение
- 6) Улучшает качество при добавлении редких айтемов (!)
- 7) Если юзер сделал новую интеракцию – можем обновить рекомендации без дообучения
- 8) Может спокойно работать в онлайнe при latency 50 мс под нагрузкой 200 rps с ~10к каталогом
- 9) Можно искать похожие айтемы через его матрицу взаимодействий

Результаты EASE

- 1) Наш коллега из Т-банка занял ~20/2к мест в соревновании по recsys на Kaggle, где EASE была основной моделью, размер каталога ~90к айтемов
- 2) Я с коллегами выиграл (1/~20 место) хакатон по рекомендациям от Ленты, просто обучив EASE
- 3) Мы с коллегами заняли 2/~20 мест в хакатоне Цифровой Прорыв, обучив EASE

Недостатки EASE

- 1) $O(I^3)$ – сложность вычисления обратной матрицы. Убрать непопулярные айтемы. Используйте scipy!
- 1) Не учитывает последовательность данных
- 2) Не работает с негативными действиям от пользователя (просмотр без клика)

Результаты EASE

Revisiting the Performance of iALS on Item Recommendation Benchmarks

Steffen Rendle
Google
Mountain View, USA
srendle@google.com

Li Zhang
Google
Mountain View, USA
liqzhang@google.com

Walid Krichene
Google
Mountain View, USA
walidk@google.com

Yehuda Koren
Google
Haifa, Israel
yehuda@google.com

Table 2: Quality results on the ML20M and MSD benchmark sorted by Recall@20 scores.

Dataset	Method	Recall@20	Recall@50	NDCG@100	Result from
ML20M	RecVAE [25]	0.414	0.553	0.442	[25]
	H+Vamp (Gated) [14]	0.413	0.551	0.445	[14]
	RaCT [18]	0.403	0.543	0.434	[18]
	Mult-VAE [17]	0.395	0.537	0.426	[17]
	LambdaNet [4]	0.395	0.534	0.427	[25]
	iALS	0.395	0.532	0.425	our result
	EASE [26]	0.391	0.521	0.420	[26]
	CDAE [28]	0.391	0.523	0.418	[17]
	Mult-DAE [17]	0.387	0.524	0.419	[17]
	SLIM [19]	0.370	0.495	0.401	[17]
	iALS	0.363	0.502	0.393	prev. result [13]
	iALS	0.360	0.498	0.386	prev. result [17]
	WARP [27]	0.314	0.466	0.341	[25]
MSD	Popularity	0.162	0.235	0.191	[26]
	EASE [26]	0.333	0.428	0.389	[26]
	iALS	0.309	0.415	0.368	our result
	RecVAE [25]	0.276	0.374	0.326	[25]
	RaCT [18]	0.268	0.364	0.319	[18]
	Mult-VAE [17]	0.266	0.364	0.316	[17]
	Mult-DAE [17]	0.266	0.363	0.313	[17]
	LambdaNet [4]	0.259	0.355	0.308	[25]
	iALS	0.211	0.312	0.257	prev. result [17]
	WARP [27]	0.206	0.302	0.249	[25]
	CDAE [28]	0.188	0.283	0.237	[17]
	Popularity	0.043	0.068	0.058	[26]

У EASE есть развитие

- 1) $O(I^3)$ – сложность вычисления обратной матрицы. Есть модификации SARSA, ELSA, там лучше
- 2) Не учитывает последовательность данных. Есть модификация SLIST
- 3) Не работает с негативными действиям от пользователя (просмотр без клика)

Scalable Linear Shallow Autoencoder for Collaborative Filtering

Vojtěch Vančura
vancurv@fit.cvut.cz
Czech Technical University in Prague
Prague, Czech Republic

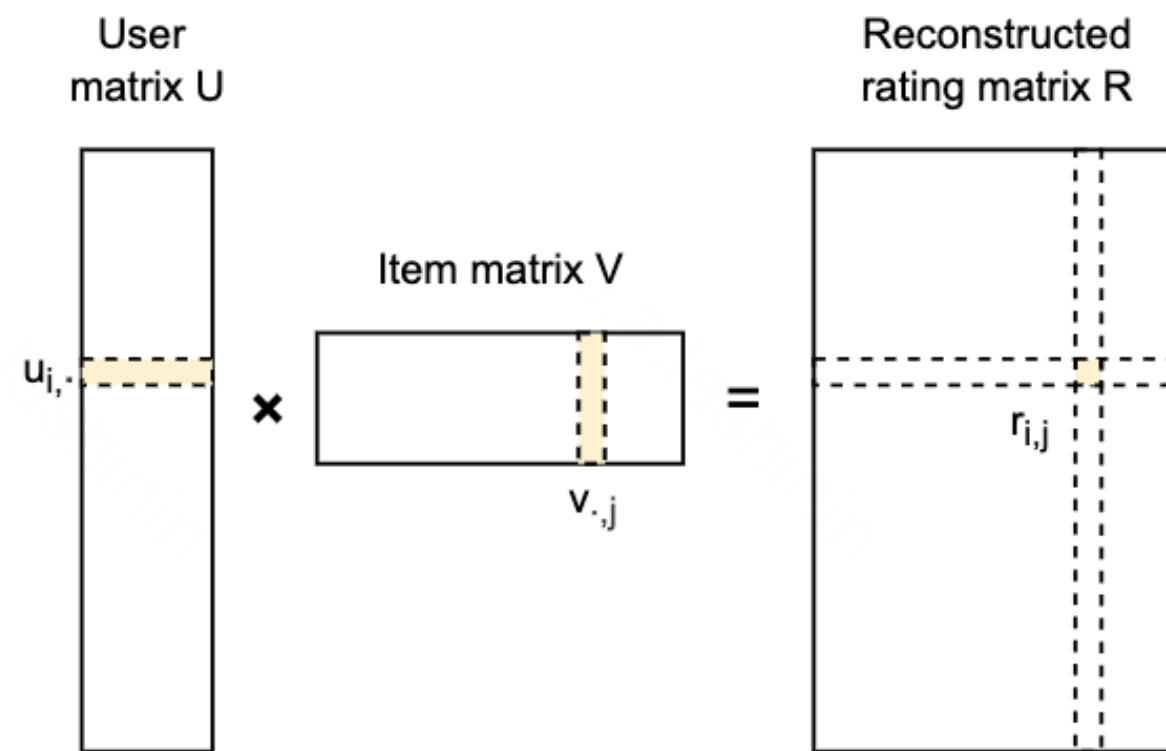
Petr Kasalický
kasalpe1@fit.cvut.cz
Czech Technical University in Prague
Prague, Czech Republic

Rodrigo Alves
rodrigo.alves@fit.cvut.cz
Czech Technical University in Prague
Prague, Czech Republic

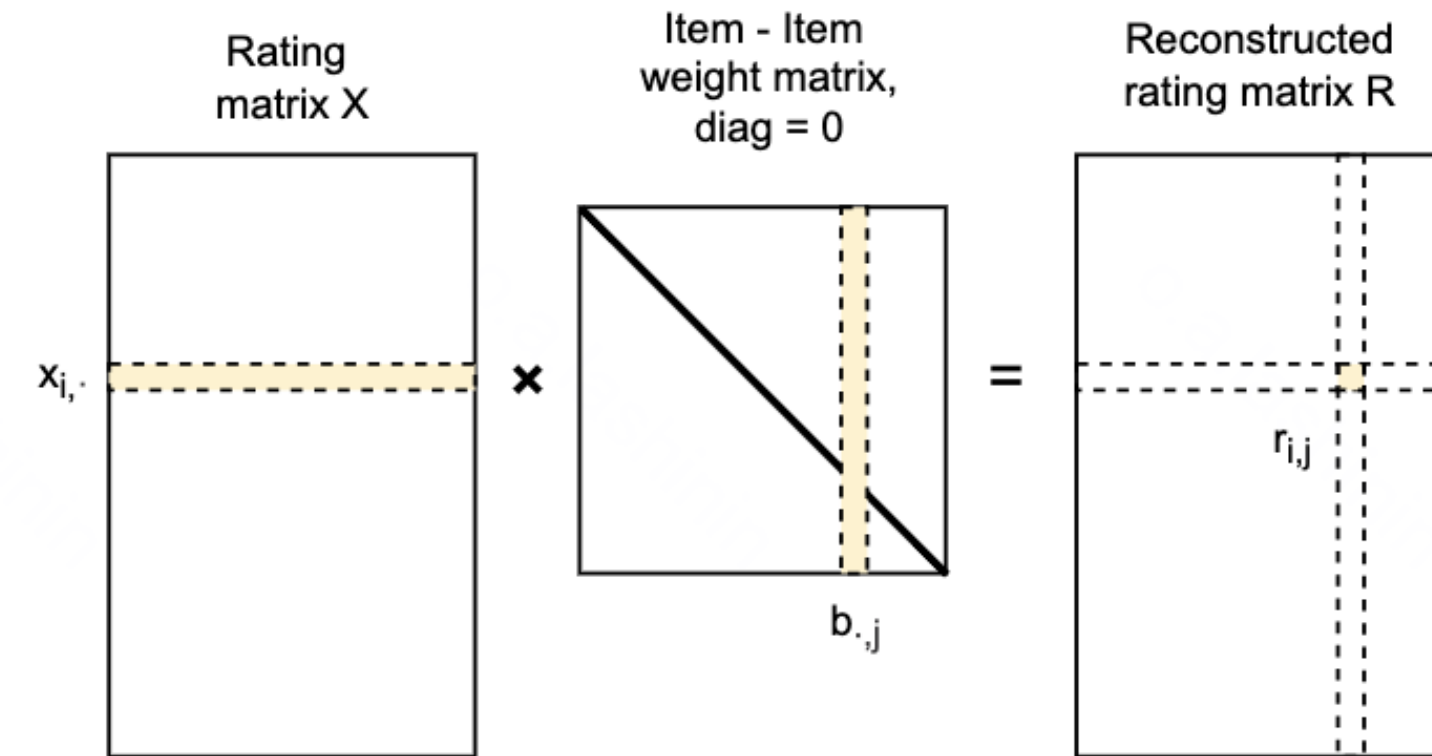
Pavel Kordík
pavel.kordik@fit.cvut.cz
Czech Technical University in Prague
Prague, Czech Republic



Matrix Factorization



EASE, SLIM



ELSA

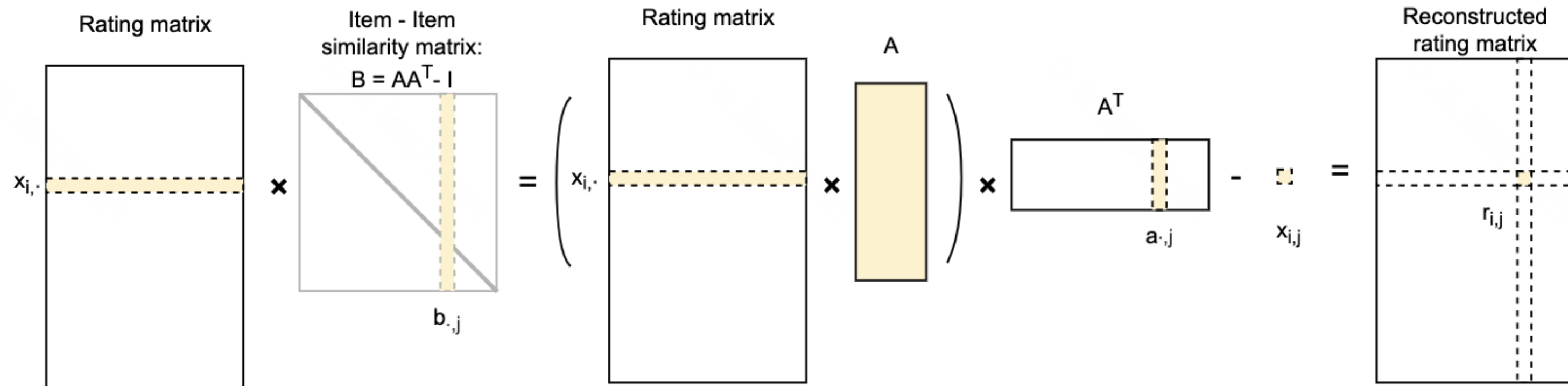
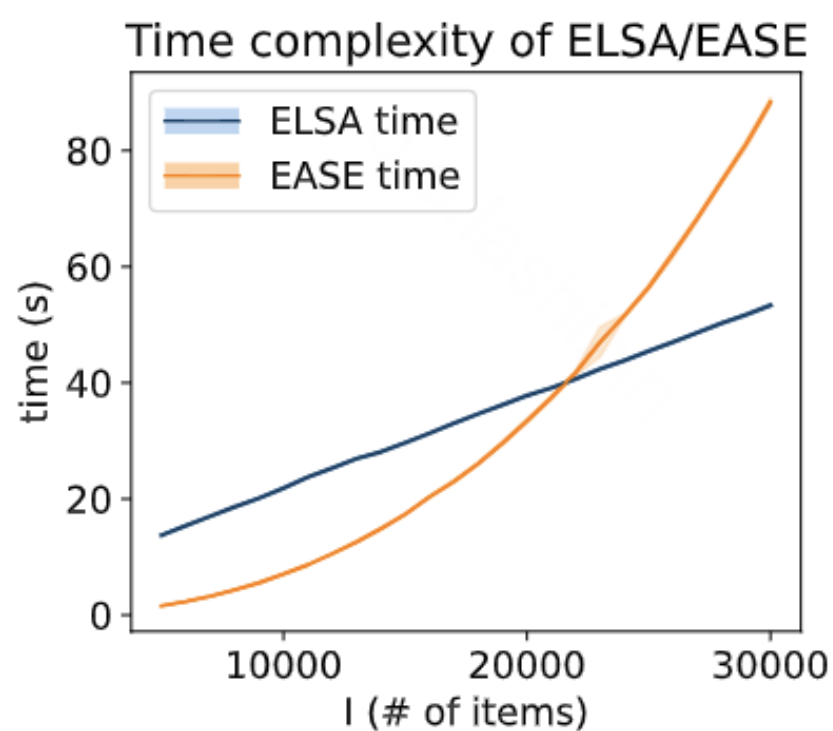
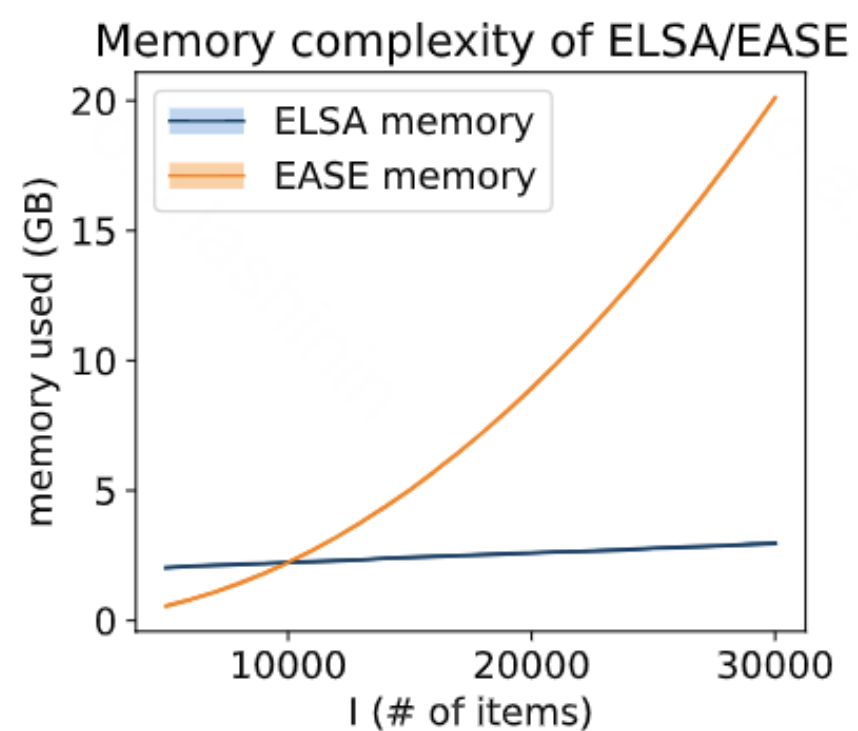


Table 1: Overall results. $N@k$ stands for $NDCG@k$, $R@k$ for $Recall@k$

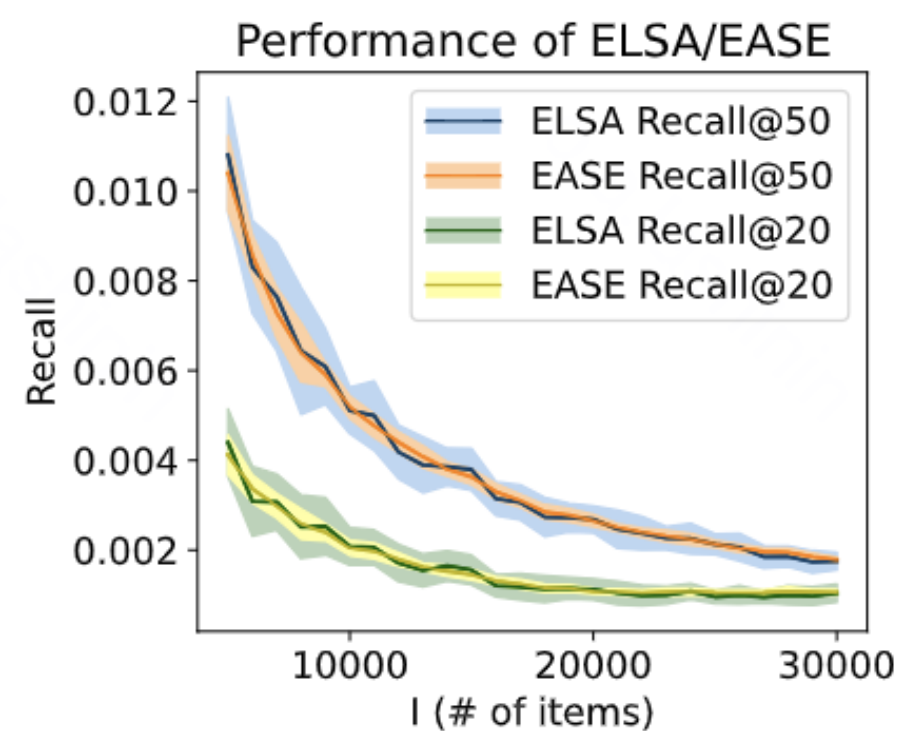
	MovieLens20M			Netflix Prize			Goodbooks-10k		
	$N@100$	$R@20$	$R@50$	$N@100$	$R@20$	$R@50$	$N@100$	$R@20$	$R@50$
Top popular	0.194	0.165	0.240	0.159	0.117	0.175	0.138	0.079	0.135
KNN	0.392	0.361	0.487	0.336	0.305	0.382	0.355	0.227	0.340
MF	0.396	0.366	0.501	0.363	0.330	0.416	0.363	0.237	0.343
EASE	0.425	0.394	0.519	0.391	0.360	0.442	0.495	0.350	0.491
ELSA	0.426	0.399	0.530	0.393	0.362	0.445	0.492	0.345	0.489



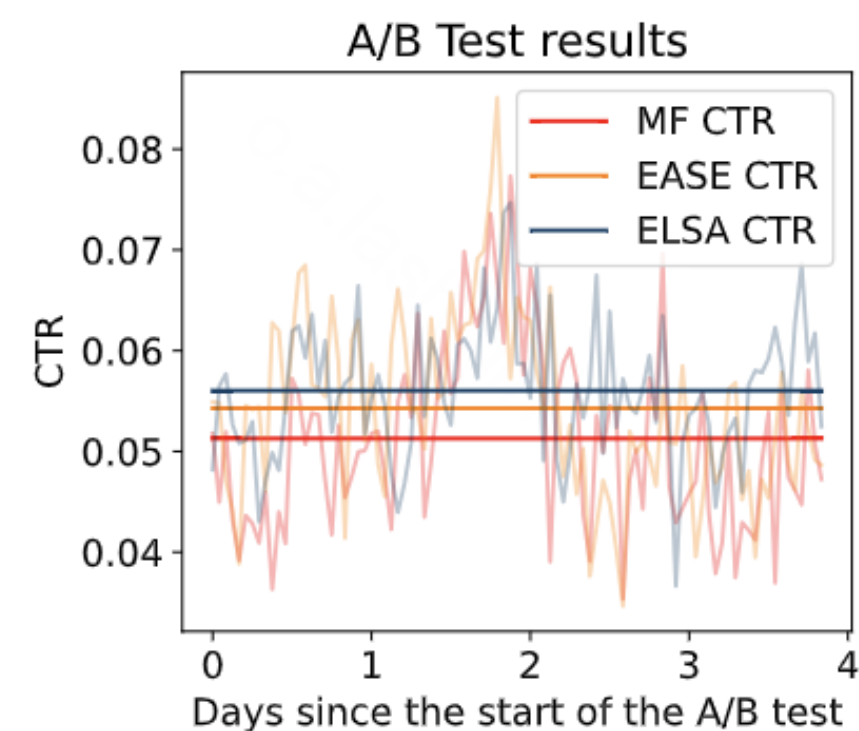
(a)



(b)



(c)



(d)

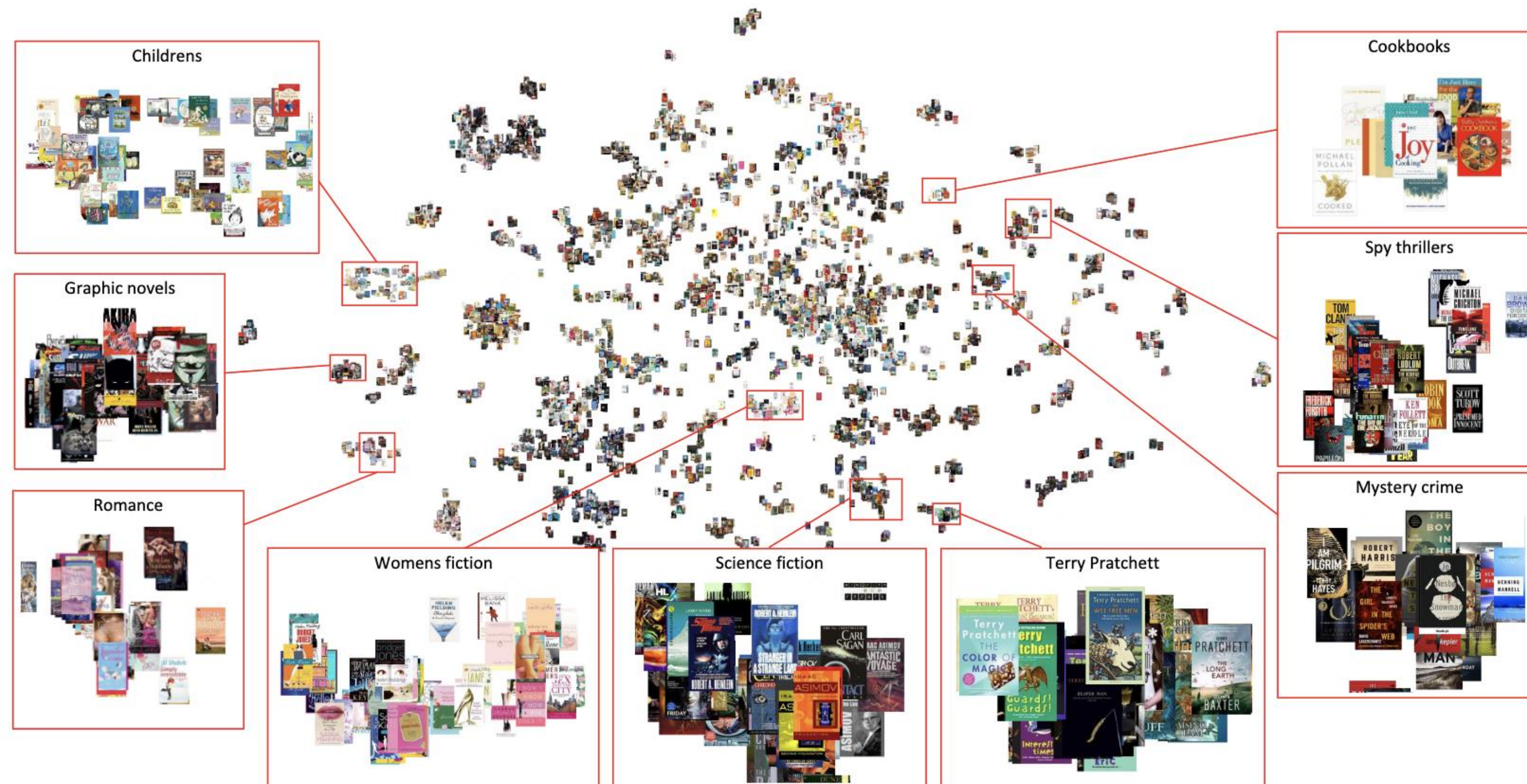


Figure 3: ELSA latent space trained on Goodbooks-10k dataset, projected to 2-d space with t-SNE. Zoomed clusters are named with the most popular tag of the books in the cluster.

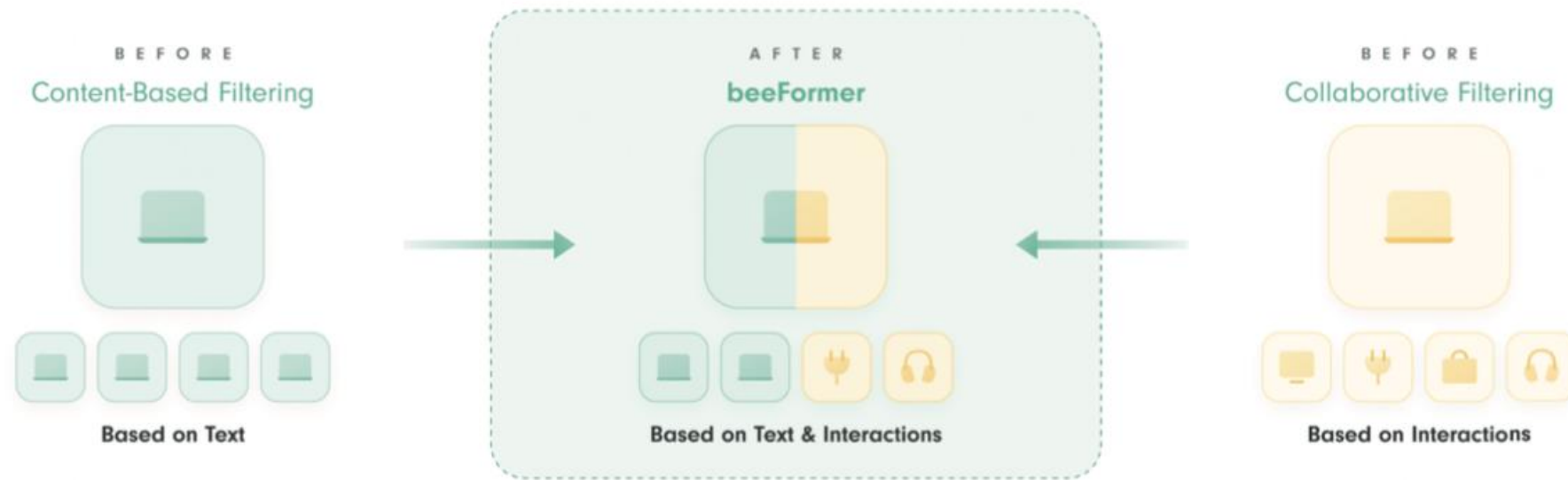
beeFormer: Bridging the Gap Between Semantic and Interaction Similarity in Recommender Systems

Vojtěch Vančura^{1,2}, Pavel Kordík^{1,2} and Milan Straka³

Recombee¹

Czech Technical University in Prague²

Charles University, Faculty of Mathematics and Physics Institute of Formal and Applied Linguistics³



Introduction

Method

We train a sentence transformer model in three steps:

- We compute matrix A from text side information without tracking gradients for optimized model
- Than we compute loss (1) with respect to A and create gradient checkpoint
- We optimize weights of sentence transformer model using gradient accumulation

$$L = \left\| \text{norm}(X_u) - \text{norm}(X_u(AA^T - I)) \right\|_F^2 \quad (1)$$

Time-Split Scenario

Dataset Method	Model	R@20	R@50	N@100
zero-shot CBF	all-mpnet-base-v2	0.0218	0.0336	0.0193
	nomic-embed-text-v1.5	0.0387	<u>0.0560</u>	<u>0.0320</u>
	bge-m3	<u>0.0398</u>	0.0546	0.0313
	Llama-goodbooks-mpnet	0.0649	0.0931	0.0515
	Llama-goodlens-mpnet	0.0617	0.0891	0.0492
supervised CF	KNN	0.0370	0.0562	0.0303
	ALS MF	0.0344	0.0580	0.0313
	ELSA	0.0367	0.0628	0.0346
CBF	SANSA	<u>0.0421</u>	<u>0.0678</u>	<u>0.0362</u>
	Llama-amazbooks-mpnet	0.0706	0.1045	0.0571

